Plano Técnico — Blossom Inbox

> *Plataforma de atendimento omnichannel customizável. Alternativa enxuta e extensível ao Chatwoot, construída sob medida para as empresas da holding Blossom Boost e preparada para ser comercializada como SaaS no futuro.*

Sumário

1. Visão Geral do Produto
2. Escopo Funcional
3. Personas e Jornadas
4. Arquitetura Técnica
5. Stack Tecnológica
6. Modelo de Deployment
7. Schema do Banco de Dados
8. Estrutura do Monorepo
9. Integrações de Canais
10. Sistema de Módulos Customizáveis
11. API REST, WebSocket e SDK
12. Sistema de Eventos e Automações
13. Wireframes e UX
14. Infra, Deploy e Provisioning
15. Segurança
16. Observabilidade e Monitoramento
17. Performance e Escalabilidade
18. Boas Práticas de Engenharia
19. Migração e Import de Dados
20. Modelo de Negócio e Go-to-Market
21. Roadmap em Fases
22. Estimativa de Esforço
23. Riscos e Mitigações
24. Decisões Pendentes
25. Anexos

---

1. Visão Geral do Produto

1.1. O que é

O **Blossom Inbox** é uma plataforma de atendimento omnichannel que centraliza conversas de múltiplos canais (WhatsApp, email, Instagram DM, Messenger, Telegram, WebChat) em um inbox unificado, com suporte a múltiplos atendentes, handoff bot↔humano, e — principal diferencial — **extensibilidade total via módulos customizáveis por cliente**.

1.2. Proposta de valor

- **Enxuto:** somente as features que as empresas da holding realmente usam. Custo de hospedagem e manutenção reduzido em comparação com Chatwoot self-hosted.
- **Customizável:** cada cliente pode ter abas, telas, actions e fluxos específicos, desenvolvidos pela equipe Blossom.
- **Multi-canal real:** bot via webhook, canais reais centralizados, com assign dinâmico entre atendentes.
- **Pronto para SaaS:** arquitetura desde o início preparada para comercialização futura (instâncias isoladas por cliente).

1.3. Público-alvo

Fase 1 — uso interno: empresas da holding Blossom Boost (EverydayFit, operação de logística, comercial da Blossom, etc.)

Fase 2 — comercialização externa: PMEs que precisam de atendimento WhatsApp+email+chatbot sem pagar caro por ferramentas gringas (Intercom, Zendesk) ou lidar com complexidade do Chatwoot.

1.4. Diferenciais-chave vs. Chatwoot

Feature	Chatwoot	Blossom Inbox
Inbox omnichannel	□	□
Multi-atendente + assign dinâmico	□	□
Bot via webhook	□	□
Abas/telas 100% customizáveis por cliente	×	□
Unificação de contato cross-canal (mesma pessoa em WA+email)	×	□ (Fase 3)
Data enrichment	×	□ (Fase 3)
Stack enxuta (Node/TS)	× (Rails pesado)	□
Instância isolada por cliente	Complicado	□ nativo

---

2. Escopo Funcional

Todas as features da transcrição do patrão estão contempladas abaixo, divididas entre **Core** (obrigatório) e **Extensões** (por cliente).

2.1. Core — Inbox Omnichannel

- **F1. Inbox unificado** com filtros por canal, status, atendente, tag
- **F2. Visualização de conversa** com histórico completo, indicação do canal de origem, e metadados da mensagem
- **F3. Envio de resposta** com suporte a texto, mídia (foto, áudio, vídeo, documento) e emojis
- **F4. Private Notes** — mensagens internas invisíveis para o contato, visíveis apenas para atendentes
- **F5. Status de conversa:** `open`, `pending`, `resolved`, `snoozed`
- **F6. Indicadores de digitação** e confirmação de leitura (onde suportado pelo canal)

2.2. Core — Canais Suportados (Fase 1)

- **C1. WhatsApp** via Twilio (crítico)
- **C2. Email** via IMAP + SMTP (ou provedor tipo Postmark/Resend — a decidir)
- **C3. WebChat Widget** — snippet HTML que o cliente cola no site dele

2.3. Canais Suportados (Fase 2)

- **C4. Instagram DM** via Twilio/Meta
- **C5. Messenger** via Twilio/Meta
- **C6. Telegram** via Bot API (nativo, não precisa Twilio)

2.4. Core — Gestão de Atendentes

- **A1. Autenticação** (email/senha, com 2FA opcional)
- **A2. Papéis:** `admin`, `supervisor`, `agent`
- **A3. Assign de conversa** — manual ou automático via regras
- **A4. Assign dinâmico** — conversa transfere entre atendentes mantendo histórico
- **A5. Filtros "mine" / "unassigned" / "all"**
- **A6. Indicador de presença** (online/offline/away)

2.5. Core — Bot / Automação

- **B1. Bot como webhook** — plataforma envia cada mensagem pro endpoint configurado, bot responde com payload JSON (mensagem, comandos de assign, actions, etc.)
- **B2. Handoff bot→humano** via comando do bot (`{ "action": "handoff", "team": "suporte" }`)
- **B3. Handoff humano→bot** (devolver conversa pro bot)
- **B4. Bot como entidade first-class** — aparece no assign como "usuário" normal

2.6. Core — Contatos

- **CT1. Listagem de contatos** com busca e filtros
- **CT2. Perfil de contato** com campos fixos (nome, foto, telefone, email, canais) e **campos customizáveis por cliente**
- **CT3. Histórico completo** de conversas do contato em todos os canais
- **CT4. Sistema de tags/labels** multi-tag por contato

2.7. Core — Actions Customizáveis

- **AC1. Botões de Action** no painel lateral do contato — cada um dispara um webhook configurável
- **AC2. Actions declaradas no módulo** do cliente (cancelar pedido, registrar devolução, aplicar cupom, etc.)
- **AC3. Suporte a formulários** antes de disparar a action (ex: pedir motivo do cancelamento)
- **AC4. Log de actions executadas** (auditoria)

2.8. Core — Dashboard

- **D1. Métricas por período:** volume de mensagens recebidas/enviadas, conversas abertas/resolvidas, tempo médio de resposta
- **D2. Métricas por atendente:** mensagens respondidas, tempo médio, conversas resolvidas

- **D3. Métricas por canal**
- **D4. Export CSV** dos dados

2.9. Core — Campanhas (Fase 2)

- **CP1. Criação de campanha** com seleção de inbox, segmentação por tag, e template de mensagem
- **CP2. Envio em massa** (respeitando limites de rate do canal)
- **CP3. Relatório de entrega** (entregue, lido, respondido, erro)
- **CP4. Disparo agendado**
- **Obs:** pra WhatsApp em massa usar **templates aprovados pela Meta** via Twilio, não bypass via Puppeteer (risco de ban permanente e não escalável para SaaS).

2.10. Extensões — Abas Customizáveis

- **E1. Sistema de plugin/módulo** — cada cliente tem um módulo em `modules/<cliente>/`
- **E2. Módulo declara:** abas novas no menu lateral, rotas backend, componentes frontend, actions, campos customizados de contato
- **E3. Exemplos reais:**
  - Logística: aba "Despacho" com formulário de erro + upload de foto
  - Tênis (pai do Sheiba): aba "Catálogo" com CRUD de produtos pro chatbot recomendar

2.11. Core — Produtividade do Atendente

- **P1. Canned Responses (Respostas Rápidas)** — biblioteca de respostas salvas com atalho `/`. Ex: digitar `/boasvindas` expande pra "Olá {nome}, como posso ajudar?" com merge tags.
- **P2. Mentions** — em Private Notes, atendente pode digitar `@joao` pra mencionar outro atendente, que recebe notificação.
- **P3. Keyboard Shortcuts** — atalhos essenciais (`j/k` navegar conversas, `r` responder, `n` private note, `a` assign, `e` resolver, `g i` ir pro inbox, etc.). Documentados em modal `?`.
- **P4. Search Global** — busca unificada em conversas, contatos e mensagens (Postgres full-text search + `tsvector`).
- **P5. Reply / Quote** — citar mensagem anterior na resposta (onde o canal suporta nativamente).
- **P6. Bulk Actions** — selecionar múltiplas conversas/contatos e aplicar: tag, assign, resolver, snooze.
- **P7. Scheduled Messages** — atendente agenda envio de mensagem pra horário futuro.
- **P8. Conversation Snooze** — "silenciar" conversa por X horas/dias, reabre automaticamente.
- **P9. Message Reactions** — reagir a mensagens internamente (emoji no historico, visível só aos atendentes).
- **P10. Draft autosave** — rascunho de resposta é salvo automaticamente, sobrevive reload/troca de conversa.
- **P11. Link Preview** — mensagens com URLs mostram preview (Open Graph).

2.12. Core — Gestão de Times

- **T1. Teams** — agrupamento de atendentes (ex: "Suporte", "Vendas", "Financeiro").
- **T2. Assign pra Team** — conversa fica disponível pra qualquer membro do time pegar ("pool").
- **T3. Round-robin assign** — distribuição automática balanceada dentro de um team.
- **T4. Skill-based routing** (Fase 2) — tags de skill nos atendentes, conversa roteada pro mais apto.
- **T5. Team-specific canned responses** — biblioteca de respostas por time.

2.13. Core — SLA, Horário Comercial e Auto-Reply

- **S1. Horário comercial configurável** por inbox (dias da semana, horários, feriados, timezone).
- **S2. Auto-reply fora de horário** — mensagem automática informando horário de atendimento.
- **S3. SLA configurável** — metas de tempo de primeira resposta e resolução por inbox/priority.
- **S4. Indicador visual de SLA** — cor na conversa (verde/amarelo/vermelho) conforme se aproxima do deadline.
- **S5. Relatório de cumprimento de SLA** no dashboard.

2.14. Core — Regras de Automação e Escalation

- **AU1. Rule engine declarativo** — "SE conversa está `pending` por > 30 min E tag = `urgente` ENTÃO assign pra supervisor + notificar".
- **AU2. Triggers disponíveis:** nova mensagem, conversa criada, conversa atribuída, tempo sem resposta, tag adicionada, evento custom de módulo.
- **AU3. Actions disponíveis:** assign, reassign, mudar status, aplicar tag, enviar mensagem, disparar webhook, executar macro.
- **AU4. Macros** — sequência nomeada de ações que atendente pode disparar com um clique (ex: "Pedido enviado": aplica tag `enviado`, envia mensagem template, resolve conversa).

2.15. Core — Notificações

- **N1. In-app notifications** — sino no header, lista notificações não lidas (nova mensagem, mention, assign).
- **N2. Browser Push Notifications** (Web Push) — funciona mesmo com aba fechada.
- **N3. Desktop notifications** via Notification API do browser.
- **N4. Email notifications** — resumo diário de conversas não respondidas, opt-in.
- **N5. Preferências por atendente** — controla quais eventos geram notificação e qual canal.

2.16. Core — CSAT e Feedback

- **CS1. Pesquisa CSAT pós-resolução** — ao resolver conversa, sistema envia mensagem com escala 1-5.
- **CS2. Coleta multi-canal** — funciona em WhatsApp, email, webchat.
- **CS3. NPS opcional** — survey de Net Promoter Score configurável.
- **CS4. Dashboard CSAT** — média por atendente, inbox, período.
- **CS5. Comentário qualitativo** — atendente pode ler o comentário do contato.

2.17. Core — Moderação e Blocklist

- **M1. Blocklist** — lista de telefones/emails bloqueados, mensagens descartadas automaticamente.
- **M2. Spam filter pra email** — integração com provedor ou regras simples.
- **M3. Flagging de contato** — atendente marca contato como "abusivo" pra supervisor revisar.
- **M4. Rate limit por contato** — limitar quantas mensagens um contato pode enviar em janela de tempo (anti-spam).

2.18. Core — API Pública e Webhooks de Saída

- **AP1. API REST pública** — endpoints autenticados por API Key pra integrações externas (CRM do cliente, BI, etc.).
- **AP2. API cobre:** CRUD de contatos, conversas, mensagens, tags; envio programático de mensagem.

- **AP3. Webhooks de saída** — cliente configura URL que recebe eventos do sistema (`conversation.resolved`, `contact.created`, etc.).
- **AP4. Assinatura HMAC** — toda entrega webhook é assinada; retry exponencial em caso de falha.
- **AP5. Documentação OpenAPI** — Swagger UI público.

2.19. Core — Mobile Experience

- **MB1. Responsive design** — UI totalmente funcional em tablet e celular.
- **MB2. PWA** — instalável, offline básico (ver conversas já carregadas).
- **MB3. Push notifications mobile** via Web Push (funciona em Android, iOS 16.4+).
- **MB4. App nativo (Fase 4+)** — React Native compartilhando lógica com Next.js, se demanda justificar.

2.20. Core — Audit Log e Transparência

- **AL1. UI de audit log** pra admin — filtros por usuário, ação, período.
- **AL2. Eventos auditados:** login/logout, assign, resolve, mudança de config, execução de action, envio de campanha, mudança de permissão.
- **AL3. Export CSV** do audit log.
- **AL4. Retenção configurável** (default 2 anos).

2.21. Features Pós-MVP (Fase 3 e 4)

- **U1. Unificação cross-canal de contatos** — detectar que o WhatsApp +55 X e o email joão@x.com são a mesma pessoa. Abordagem híbrida:
  - **Heurística determinística:** match por email/telefone explícito
  - **Heurística fuzzy:** nome similar + mesma janela temporal
  - **IA:** embedding de nome + contexto de conversa, similarity search
  - **Confirmação manual:** atendente aprova merge sugerido
- **U2. Data enrichment** — buscar dados externos (redes sociais, CNPJ via ReceitaWS, etc.) a partir de email/telefone.
- **U3. IA nativa** (usando Claude API com prompt caching):
  - Sugestão de resposta contextual
  - Sumarização de conversa longa
  - Detecção de intenção/sentimento
  - Auto-tagging de conversas
  - Tradução em tempo real (multi-idioma)
- **U4. Integrações nativas com CRMs** — HubSpot, Pipedrive, RD Station, Salesforce.
- **U5. Integração com e-commerce** — Shopify, Nuvemshop, WooCommerce (ver pedidos do contato direto).
- **U6. WeChat** — canal adicional (mencionado pelo patrão).
- **U7. Voice / Calls** — click-to-call via Twilio Voice, transcrição automática.
- **U8. Co-browsing / Screen share** no WebChat — atendente vê tela do visitante.
- **U9. AI Agent** — bot full autônomo resolvendo conversas fim-a-fim com tool use.

---

3. Personas e Jornadas

Entender os usuários do sistema é crítico pra priorizar features e desenhar fluxos.

3.1. Persona: Ana — Atendente de Suporte

- **Contexto:** atende 40-80 conversas/dia, maioria WhatsApp e email
- **Ferramentas hoje:** planilha + WhatsApp Web em 3 abas + Gmail
- **Dores:** perde mensagens entre canais, copia/cola respostas, sem handoff ao sair de férias
- **Necessidades no Blossom Inbox:** tudo num lugar, canned responses, private notes, atalhos de teclado, notificações não spammy

3.2. Persona: Bruno — Supervisor

- **Contexto:** gerencia time de 6 atendentes, responde pelas métricas
- **Dores:** não sabe quem tá sobrecarregado, SLA invisível, conversas travadas descobrem tarde
- **Necessidades:** dashboard por atendente, alertas de SLA, reassign fácil, audit log, CSAT por pessoa

3.3. Persona: Carla — Admin / TI do Cliente

- **Contexto:** instala, configura, integra com ERP do cliente
- **Dores:** webhooks chatos, integração difícil, onboarding de atendente lento
- **Necessidades:** settings UI boa, docs de API claras, bulk import, SSO (Fase 3+), audit

3.4. Persona: Daniel — Dev Blossom (cria módulos custom)

- **Contexto:** dev interno que cria módulo pra cada cliente
- **Dores potenciais:** SDK não tipado, sem hot-reload, sem staging
- **Necessidades:** SDK TypeScript, CLI `blossom module init`, hot-reload, exemplos, CI pra testes de módulo

3.5. Persona: Eduardo — Cliente Final (contato)

- **Contexto:** usuário do WhatsApp conversando com empresa cliente
- **Dores:** respostas lentas, repetir contexto ao trocar canal, bot burro sem handoff
- **Necessidades (refletidas na UX do negócio):** resposta rápida, histórico preservado no reassign, handoff limpo, CSAT opcional

3.6. Persona: Felipe — Dono do Produto

- **Contexto:** vai usar internamente, vender depois
- **Dores estratégicas:** infra crescer sem margem, churn por UX ruim, customização virar serviço caro sem escala
- **Necessidades:** métricas por tenant, módulos reutilizáveis, observabilidade central de todas instâncias

3.7. Jornadas Críticas

3.7.1. Responder conversa WhatsApp entrante

```
1. Contato envia WhatsApp
2. Twilio → webhook Blossom (< 500ms)
3. Persiste mensagem + emite WebSocket event
4. Badge de contagem sobe na UI dos atendentes com acesso
5. Ana clica, histórico abre completo
6. Ana digita (ou `/boasvindas` expande canned response)
7. Envia → Twilio → Meta → contato
8. Status "enviada" → "entregue" → "lida"
```

SLA técnico: recebimento→render na tela do atendente: p95 < 2s.

3.7.2. Bot atende e escala pra humano

```
1. Mensagem chega, inbox tem bot assigned
2. Worker → webhook do bot
3. Bot responde { "message": "Qual seu pedido?" }
4. Sistema envia ao contato
5. Contato: "cancela pedido 123"
6. Bot tenta via API cliente, falha por regra
7. Bot responde { "action": "handoff", "team": "suporte", "note": "Cancelamento precisa aprovação manual." }
8. Sistema cria Private Note, aplica tag, reassigna ao team
9. Round-robin escolhe atendente do Suporte
10. Atendente lê note, executa action custom "Cancelar (manual)"
```

3.7.3. Despachante da logística registra erro

```
1. João (despachante) abre Blossom, clica aba "Despacho" (módulo custom)
2. Formulário: seleciona contato, tipo de erro, foto
3. Submit → backend do módulo valida, registra, dispara webhook ERP
4. Cria mensagem automática na conversa: "Registramos sua ocorrência #42"
5. Supervisor recebe notificação in-app
```

3.7.4. Provisionamento de novo cliente

```
1. Felipe: `blossom tenant create acme`
2. Script: cria DB, DNS subdomain, deploy Coolify, seed user admin
3. < 5 min: URL + senha inicial via email/Slack
4. Logs em docs/tenants/acme.log
5. Módulos custom carregam via ENABLED_MODULES=acme-logistica
```

---

4. Arquitetura Técnica

4.1. Visão de Componentes

```
┌──────────────────────────────────────────────────────────────┐
│                    Instância do Cliente X                    │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────────┐   │
│  │  Frontend    │  │   Backend    │  │   Worker Queue   │   │
│  │  (Next.js)   │◄─┤  (Fastify)   │◄─┤   (BullMQ)       │   │
│  └──────┬───────┘  └──────┬───────┘  └──────┬───────────┘   │
│         │                 │                 │               │
│         │          ┌──────▼──────┐  ┌──────▼──────┐        │
│         └──────────►│  Postgres   │  │    Redis    │        │
│         WebSocket  └─────────────┘  └─────────────┘        │
│                                                              │
│  ┌──────────────────────────────────────────────────────┐   │
│  │        Módulos Custom (modules/<cliente>/)           │   │
│  └──────────────────────────────────────────────────────┘   │
└──────────────────────────────────────────────────────────────┘
            │                  │                  │
      ┌─────▼─────┐      ┌─────▼─────┐      ┌─────▼─────┐
      │  Twilio   │      │   IMAP/   │      │  Object   │
      │ (WA/IG/   │      │   SMTP    │      │  Storage  │
      │ Messenger)│      │           │      │ (R2/S3)   │
      └───────────┘      └───────────┘      └───────────┘
```

4.2. Fluxo de Mensagem Recebida (ex: WhatsApp)

```
1. Contato envia mensagem no WhatsApp
2. Twilio recebe e envia webhook para /webhooks/twilio/whatsapp
3. Backend valida assinatura Twilio, cria/atualiza Contact e Conversation
4. Persiste Message no Postgres
5. Upload de mídia (se houver) pra Object Storage em background (BullMQ)
6. Emite evento via WebSocket para atendentes conectados
7. Se conversa está com bot assigned:
   a. Worker envia webhook pro bot do cliente
   b. Bot responde com { message, action? }
   c. Resposta é persistida, enviada de volta ao Twilio, e refletida no inbox
8. Se conversa está com humano: agente recebe notificação, responde via UI
```

4.3. Fluxo de Action Customizada

```
1. Atendente clica no botão "Cancelar Pedido" no painel de contato
2. Frontend abre modal com formulário declarado pelo módulo
3. Atendente preenche e confirma
4. Backend registra log de auditoria e dispara webhook pro sistema do cliente
5. Sistema do cliente processa e responde (sucesso/erro)
6. Resultado é exibido ao atendente
7. Opcionalmente, cria Private Note automática na conversa
```

4.4. Princípios Arquiteturais

1. **Modularidade agressiva** — core nunca depende de módulo específico
2. **Event-driven** — operações pesadas (envio de mensagem, upload de mídia, envio de campanha) são sempre assíncronas via fila
3. **Idempotência** — webhooks são idempotentes (Twilio pode reenviar)
4. **Escalabilidade horizontal** — backend é stateless, múltiplas instâncias podem rodar atrás de load balancer
5. **Domain-Driven Design** leve — bounded contexts: `inbox`, `contacts`, `automation`, `campaigns`, `analytics`

---

5. Stack Tecnológica

5.1. Escolhas e Justificativas

Camada	Tecnologia	Por quê
Linguagem	TypeScript	Stack que a equipe domina (EverydayFit), ecossistema excelente, typesafe
Backend framework	Fastify	Rápido, TypeScript-first, validação nativa via JSON Schema/Zod, plugin system bem resolvido. API similar ao Express (ramp-up rápido). Preferido sobre NestJS pela leveza — arquitetura modular vem de disciplina de pastas, não do framework.
Frontend framework	Next.js 15 (App Router)	SSR onde faz sentido, client components pra UI dinâmica, ótimo DX
UI library	shadcn/ui + Tailwind	Componentes próprios (não lock-in), visual profissional, customizável
ORM	Drizzle ORM	Type-safe, migrations decentes, mais leve que Prisma, performance melhor
Banco	PostgreSQL 16	Universal, full-text search nativo, JSONB pra campos flexíveis
Cache/Queue	Redis 7	Cache de sessão, BullMQ pra filas, pub/sub
Fila de jobs	BullMQ	Robusto, baseado em Redis, retry/backoff nativo
WebSocket	Socket.IO	Maduro, reconexão automática, rooms (pra multi-atendente)
Auth	Better Auth (ou Lucia)	Moderno, TS-first, sem vendor lock-in
Object Storage	Cloudflare R2 (ou MinIO self-hosted)	R2 tem egress grátis (ideal pra SaaS); MinIO se quiser 100% self-hosted
Email (envio)	Resend ou Postmark	API moderna, bons logs, boa entregabilidade
Email (recepção)	IMAP próprio ou Postmark Inbound	Postmark Inbound é mais simples
WhatsApp/IG/Messenger	Twilio	Já validado pelo patrão, simplifica homologação Meta
Telegram	Telegram Bot API (nativo)	Gratuito, sem intermediário
Monitoramento	Sentry + Grafana/Prometheus	Sentry pra erros; Grafana pra métricas de infra
Logs	Pino + Loki (ou BetterStack)	Pino é o logger Node mais rápido
Testing	Vitest + Playwright	Unitário com Vitest, E2E com Playwright
Container	Docker + Docker Compose	Padrão, universal
Orquestração	Coolify ou Dokploy (self-hosted PaaS)	Gerencia deploys, alternativa open-source ao Render/Railway
CI/CD	GitHub Actions	Padrão mercado
IaC	Terraform (quando escalar pra K8s)	Opcional na Fase 1

5.2. Decisões de stack que NÃO fizemos (e por quê)

- **Rails (como Chatwoot):** equipe não domina, pesado, muita feature supérflua

- **NestJS:** estrutura opinada (DI, decorators, módulos) agrega valor em time grande, mas tem curva de aprendizado e overhead pra dev solo. Disciplina de pastas resolve tão bem quanto.
- **Express puro:** em modo manutenção desde 2019, sem TS first-class, precisa de muitas libs externas pra validação/auth. Fastify é evolução natural.
- **Supabase hospedado:** inviável com N instâncias isoladas (custo explode)
- **MongoDB:** relações muitas-pra-muitas (atendente ↔ conversas, tags ↔ contatos) ficam horríveis sem JOINs
- **Firebase:** vendor lock-in e custo cresce rápido com mensageria real-time
- **Microserviços desde o início:** overengineering — monolito modular é melhor até 50+ clientes

---

6. Modelo de Deployment

6.1. Instância Isolada por Cliente

Cada cliente (empresa da holding ou cliente SaaS externo) tem:

- Próprio container backend
- Próprio container frontend
- Próprio banco Postgres
- Próprio Redis
- Próprio subdomínio (`logistica.blossominbox.com` , `everydayfit.blossominbox.com`)
- Próprio bucket de storage (prefixo no R2/S3)
- Próprio conjunto de módulos custom

6.2. Por que instância isolada?

Vantagem	Detalhe
Isolamento total de dados	Zero risco de vazamento cross-tenant
Customização sem RLS	Módulos custom não precisam lidar com `tenant_id` em toda query
Deploy independente	Bug num cliente não derruba outros
Billing simples	Consumo mensurável por instância
Compliance facilitado	Cliente pode exigir que dados fiquem em região específica

6.3. Organização da infra

Fase 1 (MVP interno):

- 1 VPS Hetzner (CX32, ~€13/mês) com Coolify rodando todas as instâncias das empresas da holding
- Backups diários pro Backblaze B2 (baratíssimo)

Fase 2 (primeiros clientes externos):

- Cluster de 2-3 VPS com Coolify + load balancer
- Cloudflare na frente (CDN, WAF, DDoS)

Fase 3 (escala SaaS):

- Migrar pra Kubernetes (k3s self-hosted ou managed)

- Banco dedicado por tier (free/starter/pro)
- Object Storage R2 em produção

---

7. Schema do Banco de Dados

7.1. Tabelas Principais

```
-- USUÁRIOS (atendentes da plataforma)
users (
  id            UUID PK
  email         TEXT UNIQUE NOT NULL
  password_hash TEXT NOT NULL
  name          TEXT NOT NULL
  avatar_url    TEXT
  role          ENUM('admin', 'supervisor', 'agent')
  status        ENUM('online', 'offline', 'away')
  last_seen_at  TIMESTAMPTZ
  created_at    TIMESTAMPTZ DEFAULT NOW()
)

-- CONTATOS (clientes finais que conversam)
contacts (
  id              UUID PK
  name            TEXT
  email           TEXT
  phone           TEXT
  avatar_url      TEXT
  custom_fields   JSONB DEFAULT '{}'  -- campos customizáveis por cliente
  created_at      TIMESTAMPTZ DEFAULT NOW()
  updated_at      TIMESTAMPTZ DEFAULT NOW()
  INDEX (email), INDEX (phone)
)

-- IDENTIDADES POR CANAL (contato pode ter múltiplos: WA, email, IG)
contact_identities (
  id              UUID PK
  contact_id      UUID FK→contacts
  channel         ENUM('whatsapp', 'email', 'instagram', 'messenger', 'telegram', 'webchat')
  identifier      TEXT NOT NULL  -- número, email, user_id
  metadata        JSONB DEFAULT '{}'
  UNIQUE(channel, identifier)
)

-- INBOXES (canais configurados no sistema)
inboxes (
  id              UUID PK
  name            TEXT NOT NULL
  channel_type    ENUM(...)  -- mesmo enum acima
  config          JSONB  -- tokens Twilio, IMAP creds, etc. (criptografado)
  created_at      TIMESTAMPTZ DEFAULT NOW()
)

-- INBOX ↔ USUÁRIOS (quem tem acesso a qual inbox)
inbox_members (
  inbox_id        UUID FK→inboxes
  user_id         UUID FK→users
  PRIMARY KEY (inbox_id, user_id)
)

-- CONVERSAS
conversations (
  id              UUID PK
  contact_id      UUID FK→contacts
  inbox_id        UUID FK→inboxes
  assigned_to     UUID FK→users NULL  -- NULL se bot
  assigned_bot_id UUID FK→bots NULL
  status          ENUM('open', 'pending', 'resolved', 'snoozed')
  priority        ENUM('low', 'medium', 'high', 'urgent') DEFAULT 'medium'
  last_message_at TIMESTAMPTZ
  created_at      TIMESTAMPTZ DEFAULT NOW()
  resolved_at     TIMESTAMPTZ NULL
  INDEX (assigned_to, status), INDEX (inbox_id, status)
)

-- MENSAGENS
messages (
  id              UUID PK
  conversation_id UUID FK→conversations
  sender_type     ENUM('contact', 'user', 'bot', 'system')
  sender_id       UUID NULL  -- user_id se sender_type='user'
  content         TEXT
  content_type    ENUM('text', 'image', 'audio', 'video', 'document', 'location')
  media_url       TEXT NULL
  is_private_note BOOLEAN DEFAULT FALSE
  channel_msg_id  TEXT  -- ID externo (Twilio, etc.) pra idempotência
  metadata        JSONB DEFAULT '{}'
  created_at      TIMESTAMPTZ DEFAULT NOW()
  INDEX (conversation_id, created_at), UNIQUE(channel_msg_id)
)

-- TAGS
tags (
  id              UUID PK
  name            TEXT NOT NULL
  color           TEXT  -- hex
  UNIQUE(name)
```

```
)

contact_tags (
  contact_id      UUID FK→contacts
  tag_id          UUID FK→tags
  PRIMARY KEY (contact_id, tag_id)
)

conversation_tags (
  conversation_id UUID FK→conversations
  tag_id          UUID FK→tags
  PRIMARY KEY (conversation_id, tag_id)
)

-- BOTS
bots (
  id              UUID PK
  name            TEXT NOT NULL
  webhook_url     TEXT NOT NULL
  secret          TEXT NOT NULL  -- pra HMAC
  inbox_id        UUID FK→inboxes
  enabled         BOOLEAN DEFAULT TRUE
)

-- ACTIONS CUSTOMIZADAS
custom_actions (
  id              UUID PK
  module_key      TEXT  -- identificador do módulo que declarou
  name            TEXT NOT NULL
  description     TEXT
  icon            TEXT
  form_schema     JSONB  -- JSON Schema do formulário
  webhook_url     TEXT
  requires_role   ENUM('admin', 'supervisor', 'agent') DEFAULT 'agent'
)

action_logs (
  id              UUID PK
  action_id       UUID FK→custom_actions
  user_id         UUID FK→users
  contact_id      UUID FK→contacts
  conversation_id UUID FK→conversations NULL
  payload         JSONB
  response        JSONB
  status          ENUM('success', 'error')
  executed_at     TIMESTAMPTZ DEFAULT NOW()
)

-- CAMPANHAS
campaigns (
  id              UUID PK
  name            TEXT
  inbox_id        UUID FK→inboxes
  template_id     TEXT  -- Twilio template
  audience_filter JSONB  -- critérios de segmentação
  scheduled_at    TIMESTAMPTZ NULL
  status          ENUM('draft', 'scheduled', 'running', 'completed', 'failed')
  created_by      UUID FK→users
  created_at      TIMESTAMPTZ DEFAULT NOW()
)

campaign_messages (
  id              UUID PK
  campaign_id     UUID FK→campaigns
  contact_id      UUID FK→contacts
  status          ENUM('pending', 'sent', 'delivered', 'read', 'replied', 'failed')
  error           TEXT NULL
  sent_at         TIMESTAMPTZ NULL
)

-- AUDIT LOG geral
audit_logs (
  id          UUID PK
  user_id     UUID FK→users NULL
  action      TEXT        -- "conversation.assigned", "contact.updated"
  entity_type TEXT
  entity_id   UUID
  changes     JSONB
  created_at  TIMESTAMPTZ DEFAULT NOW()
  INDEX (entity_type, entity_id, created_at)
)
```

7.2. Considerações

- Todas as tabelas têm `created_at` / `updated_at` com triggers
- Soft delete (`deleted_at`) em `contacts` , `conversations` , `messages` para preservar histórico

- Índices otimizados pras queries mais comuns (inbox + status, contact + channel)
- `custom_fields` em JSONB permite módulos adicionarem dados sem alterar schema

---

8. Estrutura do Monorepo

```
blossom-inbox/
├── apps/
│   ├── backend/             # Fastify API + WebSocket
│   │   ├── src/
│   │   │   ├── modules/
│   │   │   │   ├── inbox/
│   │   │   │   ├── contacts/
│   │   │   │   ├── conversations/
│   │   │   │   ├── messages/
│   │   │   │   ├── automation/
│   │   │   │   ├── campaigns/
│   │   │   │   ├── analytics/
│   │   │   │   ├── channels/
│   │   │   │   │   ├── whatsapp/
│   │   │   │   │   ├── email/
│   │   │   │   │   ├── webchat/
│   │   │   │   │   └── telegram/
│   │   │   │   └── plugins/      # sistema de módulos custom
│   │   │   ├── common/
│   │   │   ├── config/
│   │   │   └── main.ts
│   │   ├── test/
│   │   └── Dockerfile
│   │
│   ├── frontend/            # Next.js 15
│   │   ├── app/
│   │   │   ├── (auth)/
│   │   │   ├── (dashboard)/
│   │   │   │   ├── inbox/
│   │   │   │   ├── contacts/
│   │   │   │   ├── campaigns/
│   │   │   │   ├── analytics/
│   │   │   │   └── settings/
│   │   │   └── plugins/      # rotas dinâmicas de módulos
│   │   ├── components/
│   │   ├── lib/
│   │   └── Dockerfile
│   │
│   └── worker/              # BullMQ workers (pode rodar junto do backend)
│       └── src/
│
├── packages/
│   ├── shared-types/        # tipos TS compartilhados
│   ├── sdk/                 # SDK pra módulos custom (registra abas, actions, etc.)
│   ├── ui/                  # componentes shadcn compartilhados
│   └── db/                  # Drizzle schemas + migrations
│
├── modules/                 # MÓDULOS CUSTOM POR CLIENTE
│   ├── logistica-acme/
│   │   ├── backend/
│   │   │   └── index.ts     # registra rotas, actions, webhooks
│   │   ├── frontend/
│   │   │   └── pages/       # componentes React de abas custom
│   │   └── module.config.ts # metadata (nome, cliente, versão, features)
│   ├── everydayfit/
│   └── tenis-sheiba/
│
├── infra/
│   ├── docker-compose.yml
│   ├── docker-compose.dev.yml
│   └── coolify/             # templates Coolify
│
├── scripts/
│   ├── provision-tenant.sh  # cria nova instância pra cliente
│   └── migrate.sh
│
├── docs/
│   ├── PLANO-TECNICO.md     # este documento
│   ├── ARCHITECTURE.md
│   ├── MODULE-DEVELOPMENT.md # guia de criar módulo custom
│   └── API.md
│
├── turbo.json
├── package.json             # workspace root
└── README.md
```

8.1. Gerenciador: Turborepo + pnpm workspaces

- Build paralelo
- Cache de build
- Módulos compartilham dependências

---

9. Integrações de Canais

Cada canal expõe interface `ChannelAdapter` uniforme no core:

```
interface ChannelAdapter {
  type: ChannelType;
  sendMessage(inbox, to, payload): Promise<{ externalId: string }>;
  handleWebhook(req): Promise<IncomingMessage | null>;
  downloadMedia(externalUrl): Promise<Buffer>;
  validateSignature(req): boolean;
  // Recursos opcionais
  sendTypingIndicator?(inbox, to, isTyping): Promise<void>;
  markAsRead?(inbox, externalMessageId): Promise<void>;
  deleteMessage?(inbox, externalMessageId): Promise<void>;
}
```

Isso permite trocar provedor (ex: Twilio → Meta direto) sem refatorar o core.

9.1. WhatsApp (via Twilio)

Setup inicial:

1. Criar conta Twilio, ativar WhatsApp Business
2. Conectar WABA (WhatsApp Business Account) — via Twilio Console
3. Aprovar número WhatsApp (processo Meta, 1-7 dias)
4. Criar templates de mensagem (HSM) pra mensagens iniciadas pelo negócio
5. Configurar webhook inbound no Twilio Console:
   `https://<tenant>.blossominbox.com/webhooks/twilio/whatsapp`
6. Configurar webhook de status (delivery, read):
   `https://<tenant>.blossominbox.com/webhooks/twilio/status`

Envio de mensagem:

```
// packages/backend/src/channels/whatsapp/twilio.adapter.ts
async sendMessage(inbox, to, payload) {
  const twilio = createClient(inbox.config.sid, inbox.config.token);

  const body: any = {
    from: `whatsapp:${inbox.config.number}`,
    to: `whatsapp:${to}`,
  };

  if (payload.type === 'text') {
    body.body = payload.content;
  } else if (payload.type === 'template') {
    body.contentSid = payload.templateSid;
    body.contentVariables = JSON.stringify(payload.variables);
  } else if (payload.type === 'media') {
    body.mediaUrl = payload.mediaUrl;
    body.body = payload.caption ?? '';
  }

  const msg = await twilio.messages.create(body);
  return { externalId: msg.sid };
}
```

Janela de 24h: depois de 24h sem resposta do contato, só dá pra mandar mensagem via template aprovado pela Meta. UI mostra indicador e bloqueia envio de texto livre.

Webhook entrante (payload Twilio):

```
{
  "MessageSid": "SMxxxx",
  "From": "whatsapp:+5511999999999",
  "To": "whatsapp:+551140041234",
  "Body": "Olá, quero ajuda",
  "NumMedia": "1",
  "MediaUrl0": "https://api.twilio.com/.../Media/MExxx",
  "MediaContentType0": "image/jpeg",
  "ProfileName": "João Silva"
}
```

Validação de assinatura:

```
function validateTwilioSignature(req) {
  const expected = req.headers['x-twilio-signature'];
  const url = `https://${req.hostname}${req.url}`;
  const params = req.body;  // concat ordenada de key+value
  const computed = hmacSha1(authToken, url + Object.entries(params).sort().map(([k,v]) => k+v).join(''));
  return expected === base64(computed);
}
```

Mídia: Twilio serve mídia em URL temporária (24-48h). Worker assíncrono:

1. Baixa mídia (autenticação HTTP Basic com SID+Token)
2. Upload pro R2 em path `tenants/<tenant>/media/<msg-id>/<filename>`
3. Gera thumbnail se for imagem/vídeo
4. Atualiza `messages.mediaUrl` com URL permanente do R2

Idempotência: `messages.channelMsgId = MessageSid`, UNIQUE constraint evita duplicata se Twilio reentregar webhook.

Rate limits Twilio:

- 80 msg/s por número (default, pode aumentar)
- 1000 msg/dia pra números novos (warmup gradual)

Custos (referência):

- ~US$0.005 por mensagem de sessão
- ~US$0.05 por template (varia por país, BR é mais caro)

9.2. Email

Opção A — Postmark (recomendação MVP):

Setup:

1. Criar Server em Postmark
2. Adicionar subdomínio (ex: `atendimento.cliente.com.br`) com registros DNS (SPF, DKIM)
3. Configurar Inbound Stream — Postmark gera endereço tipo `abc@inbound.postmarkapp.com`, mas melhor usar subdomínio próprio com MX apontando pra Postmark
4. Webhook Inbound: `https://<tenant>.blossominbox.com/webhooks/postmark/inbound`

Envio:

```
async sendMessage(inbox, to, payload) {
  const res = await fetch('https://api.postmarkapp.com/email', {
    method: 'POST',
    headers: {
      'X-Postmark-Server-Token': inbox.config.serverToken,
      'Content-Type': 'application/json',
    },
    body: JSON.stringify({
      From: inbox.config.fromEmail,
      To: to,
      Subject: payload.subject,
      HtmlBody: payload.html,
      TextBody: payload.text,
      MessageStream: 'outbound',
      Headers: [
        { Name: 'In-Reply-To', Value: payload.inReplyTo },  // threading
      ],
      Attachments: payload.attachments,
    }),
  });
  const data = await res.json();
  return { externalId: data.MessageID };
}
```

Recepção (webhook Postmark):

```
{
  "MessageID": "abc-123",
  "From": "joao@email.com",
  "FromName": "João Silva",
  "To": "suporte@cliente.com.br",
  "Subject": "Dúvida sobre pedido",
  "TextBody": "Gostaria de saber...",
  "HtmlBody": "<p>...</p>",
  "StrippedTextReply": "Gostaria de saber...",  // sem signature/quoted
  "MessageID": "...",
  "Headers": [
    { "Name": "In-Reply-To", "Value": "<abc@...>" },
    { "Name": "References", "Value": "<abc@...> <xyz@...>" }
  ],
  "Attachments": [...]
}
```

Threading (agrupar em conversa):

1. Se header `In-Reply-To` existe, procura `messages.channelMsgId = valor` — associa à mesma Conversation
2. Se não, procura por `subject` normalizado + `from` nas últimas 30 dias
3. Se não, cria nova Conversation

Anexos: Postmark fornece em base64 inline no webhook → upload direto pro R2, mesmo esquema do WhatsApp.

Anti-spam:

- Verificar SPF/DKIM (Postmark entrega só se passar)
- Rate limit por `from` (anti-abuse)
- Regex pra auto-replies óbvios (assunto contém "Re:", "Automatic reply", etc.) — flag de `auto-reply`

Opção B — IMAP/SMTP próprio:

Mais controle, mas muito mais complexidade:

- IMAP IDLE pra receber em tempo quase real
- SMTP com TLS
- Gerenciar bounce (retorno de email inexistente)
- Reputação do IP (warmup longo)
- Decidir: manter na Fase 3+ se algum cliente exigir infra própria

9.3. WebChat Widget

Snippet que o cliente cola no site:

```
<script>
  (function(){
    window.BlossomChat = {
      tenant: 'cliente-x',
      inbox: 'web-widget-123',
    };
    var s = document.createElement('script');
    s.src = 'https://cdn.blossominbox.com/widget.js';
    s.async = true;
    document.head.appendChild(s);
  })();
</script>
```

Widget em si (Preact, < 30KB gzipped):

- Bubble no canto inferior direito
- Expandível com lista de mensagens
- Suporte a texto, imagem, emoji
- Autenticação: token público do inbox (só permite criar contato anônimo + enviar msgs)
- Identidade persistente via `localStorage.visitorId`
- Opcional: `BlossomChat.identify({ email, name, customFields })` — link com contato já existente

Real-time: conexão WebSocket direta, sem polling.

Fluxo inbound:

1. Visitor envia msg pelo widget
2. Widget → `POST /webhooks/webchat/<inboxId>` com `visitorId + content`
3. Sistema cria Contact (se novo) com `channel: webchat`, `identifier: visitorId`
4. Cria Message, emite eventos
5. Atendente vê no inbox, responde
6. Resposta vai via WebSocket de volta pro widget

Features do widget:

- Customização de cor/logo via config do inbox
- Proactive message (auto-msg após X segundos) — Fase 2
- File upload (imagem, PDF)
- Typing indicator bidirecional
- Read receipts
- Persistência de conversa mesmo ao trocar de página (localStorage)

9.4. Instagram DM e Messenger (via Twilio)

Setup:

1. Página Facebook/Instagram conectada no Business Manager
2. Integrar com Twilio Conversations
3. Webhook pro Blossom (similar ao WhatsApp)

Diferenças:

- **Janela de 7 dias** em vez de 24h (a partir de mensagem do usuário)
- **Formato de mídia** diferente (Twilio normaliza pra JSON similar ao WA)
- **Story Replies** — Instagram envia contexto de qual story o usuário respondeu

Disponibilidade: Fase 2.

9.5. Telegram (nativo, sem Twilio)

Setup:

1. Criar Bot via @BotFather, obter token
2. Registrar webhook: `curl -X POST https://api.telegram.org/bot<TOKEN>/setWebhook -d url=https://<tenant>.blossominbox.com/webhooks/telegram/<inboxId>`

Envio:

```
async sendMessage(inbox, chatId, payload) {
  const res = await fetch(`https://api.telegram.org/bot${inbox.config.token}/sendMessage`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({
      chat_id: chatId,
      text: payload.content,
      parse_mode: 'MarkdownV2',
    }),
  });
  const data = await res.json();
  return { externalId: String(data.result.message_id) };
}
```

Vantagens:

- Gratuito
- Sem homologação Meta/Twilio
- Grupos e canais suportados (se quiser)
- API completa e estável

Desvantagens:

- Mercado BR pequeno (maioria usa WhatsApp)
- Sem templates pra outbound massa

Disponibilidade: Fase 2 (nativa, sem Twilio → mais rápida de integrar que IG/Messenger).

9.6. SMS (via Twilio, opcional)

SMS puro via Twilio também é simples:

- Útil pra OTP, alertas transacionais
- Preço ~R$0.15/msg no BR
- Incluir no core como `channel: sms` (Fase 3)

9.7. Voice (Fase 4+)

- Twilio Voice: click-to-call do painel do atendente
- Transcrição automática (Whisper/Deepgram)
- Gravação anexada à conversa
- Deep integration: conversa de voz aparece no inbox como "chamada de 3min"

9.8. Gerenciamento de Templates WhatsApp (HSM)

Mensagens iniciadas pelo negócio (fora da janela 24h) exigem template aprovado pela Meta.

Features:

- Cadastro de templates na UI do Blossom (name, categoria, conteúdo com variáveis)
- Envio para aprovação Meta via API Twilio
- Lista de templates aprovados/pendentes/rejeitados
- Ao criar campanha, só mostra templates aprovados
- Preview com merge tags

Categorias Meta:

- Authentication (OTP)
- Marketing (promoções)
- Utility (notificações transacionais)

Precificação diferente por categoria — Marketing é mais caro.

9.9. Observações sobre bypass (Puppeteer / WhatsApp Web)

O patrão mencionou possibilidade de usar biblioteca tipo Baileys/Puppeteer que conecta no WhatsApp Web do número pessoal.

Por que NÃO vamos fazer isso:

1. **Proibido pelos ToS da Meta** — ban permanente do número se detectado
2. **Não escalável** pra SaaS — cada cliente precisaria de número real, warmup manual
3. **Frágil** — quebra a cada update do WhatsApp Web
4. **Riscos legais** — cliente que usa isso está violando termos, nossa plataforma seria conivente
5. **Reputação** — cliente ganhar ban pela nossa ferramenta = churn imediato

Exceção considerável: cliente do tipo "marketing agressivo de WhatsApp" já faz isso hoje com ferramentas piratas. Poderia haver módulo separado (com disclaimer forte) que integra com `baileys` pra casos específicos. **Não recomendado pra produto principal.** Avaliação caso-a-caso.

9.10. Failure Modes e Retry

Falha	Comportamento
Twilio 5xx	Retry exponencial (10s, 30s, 2min, 10min) até 5x
Twilio 4xx	Log, marca msg como `failed`, notifica atendente (não retry)
Webhook entrante inválido	Log + 400 response (Twilio não re-envia)
Upload mídia R2 falha	Retry 3x, se falhar persist URL original Twilio (curto prazo)
Bot webhook timeout	Fallback: assigna pra fila de humanos com tag `bot-error`
Postmark bounce	Marca email do contato como `bounced`, mostra aviso na UI

---

10. Sistema de Módulos Customizáveis

Esse é o **coração da proposta de valor** do Blossom Inbox. Precisa ser bem desenhado.

10.1. O que um módulo pode fazer

- Adicionar **abas novas** no menu lateral da plataforma
- Adicionar **botões de Action** no painel do contato

- Adicionar **campos customizados** ao perfil do contato
- Adicionar **rotas backend** (REST ou WebSocket) pra lógica de negócio
- Consumir **eventos do sistema** (nova mensagem, conversa resolvida, contato criado)
- Renderizar **componentes React próprios** na UI

10.2. Anatomia de um módulo

```
modules/logistica-acme/
├── module.config.ts          # metadata declarativo
├── backend/
│   ├── index.ts              # entry point, registra no sistema
│   ├── routes.ts             # REST endpoints
│   ├── actions.ts            # handlers de Actions
│   └── events.ts             # listeners de eventos do core
├── frontend/
│   ├── pages/
│   │   └── Despacho.tsx      # componente da aba "Despacho"
│   └── components/
│       └── ErroForm.tsx
└── package.json
```

10.3. `module.config.ts` — exemplo

```typescript
import type { ModuleConfig } from '@blossom/sdk';

export default {
  key: 'logistica-acme',
  name: 'Módulo Logística ACME',
  version: '1.0.0',
  tenant: 'logistica-acme',

  tabs: [
    {
      key: 'despacho',
      label: 'Despacho',
      icon: 'TruckIcon',
      requires_role: 'agent',
      component: () => import('./frontend/pages/Despacho'),
    },
  ],

  actions: [
    {
      key: 'cancelar_pedido',
      label: 'Cancelar Pedido',
      icon: 'XCircleIcon',
      form: {
        motivo: { type: 'select', options: ['Cliente desistiu', 'Erro de preço', 'Outro'] },
        observacao: { type: 'text', required: false },
      },
      handler: 'actions/cancelarPedido',
    },
  ],

  contactFields: [
    { key: 'cpf', label: 'CPF', type: 'text' },
    { key: 'endereco', label: 'Endereço', type: 'textarea' },
  ],

  events: {
    'conversation.resolved': 'events/onResolved',
  },
} satisfies ModuleConfig;
```

10.4. Carregamento dinâmico

- **Backend:** no startup, carrega todos os módulos habilitados pra aquele tenant (variável de ambiente `ENABLED_MODULES=logistica-acme,outro`)
- **Frontend:** Next.js carrega rotas de módulo via lazy import + code splitting — só o JS do módulo do cliente vai no bundle

10.5. SDK `@blossom/sdk`

Fornece:

- Tipos (`Contact` , `Conversation` , `Message` , `ActionHandler` , etc.)
- Helpers de autenticação e contexto
- Componentes React base (Modal, Form, etc.)
- Funções pra disparar eventos e chamar API interna

10.6. Isolamento de segurança

- Módulo não pode acessar banco direto — usa SDK (camada fina, validada)
- Webhooks de actions externas têm rate limit (10 req/min default) e timeout (10s)
- Logs de auditoria de todas as ações de módulo em `audit_logs`
- Cada módulo roda no mesmo processo do core (não é sandbox real — cuidado com código próprio)
- Módulos têm acesso a `ctx.secrets` separados (não veem secrets de outros módulos)
- Namespace de tabelas próprio via `ctx.db.schema('logistica_acme')` — evita colisão

10.7. Exemplo Completo 1: Módulo "Logística ACME"

Contexto: cliente é empresa de logística. Usa Blossom Inbox pra SAC WhatsApp. Precisa de:

- Aba "Despacho" onde despachante registra erros com foto
- Action "Cancelar pedido" disparada pelo atendente
- Action "Consultar status" que busca no ERP deles e adiciona como Private Note
- Campo custom no contato: CPF, número de pedido ativo, endereço de entrega

10.7.1. Estrutura de pastas

```
modules/logistica-acme/
├── module.config.ts
├── package.json
├── tsconfig.json
├── backend/
│   ├── index.ts
│   ├── routes/
│   │   └── despacho.ts
│   ├── actions/
│   │   ├── cancelarPedido.ts
│   │   └── consultarStatus.ts
│   ├── events/
│   │   └── onConversationResolved.ts
│   └── db/
│       └── schema.ts
└── frontend/
    ├── pages/
    │   └── Despacho.tsx
    └── components/
        ├── FormErro.tsx
        └── ListaOcorrencias.tsx
```

10.7.2. `module.config.ts`

```
import { defineModule } from '@blossom/sdk';

export default defineModule({
  key: 'logistica-acme',
  name: 'Módulo Logística ACME',
  version: '1.2.0',
  tenant: 'acme-logistica',

  // Abas custom no menu lateral
  tabs: [
    {
      key: 'despacho',
      label: 'Despacho',
      icon: 'TruckIcon',
      requiresRole: 'agent',
      component: () => import('./frontend/pages/Despacho'),
    },
  ],

  // Botões de Action no painel de contato
  actions: [
    {
      key: 'cancelar_pedido',
      label: 'Cancelar Pedido',
      icon: 'XCircleIcon',
      color: 'red',
      confirmRequired: true,
      form: {
        motivo: {
          type: 'select',
          label: 'Motivo',
          required: true,
          options: [
            { value: 'cliente_desistiu', label: 'Cliente desistiu' },
            { value: 'erro_preco', label: 'Erro de preço' },
            { value: 'endereco_invalido', label: 'Endereço inválido' },
            { value: 'outro', label: 'Outro' },
          ],
        },
        observacao: { type: 'textarea', label: 'Observação', required: false, maxLength: 500 },
      },
      handler: 'cancelarPedido',  // referencia arquivo backend/actions/cancelarPedido.ts
      postExecute: {
        addPrivateNote: true,      // cria Private Note com resultado
        applyTag: 'cancelado',
      },
    },
    {
      key: 'consultar_status',
      label: 'Consultar Status do Pedido',
      icon: 'SearchIcon',
      color: 'blue',
      form: null,                     // sem formulário, executa direto
      handler: 'consultarStatus',
    },
  ],

  // Campos customizados no perfil de contato
  contactFields: [
    { key: 'cpf', label: 'CPF', type: 'text', validate: 'cpf' },
    { key: 'pedido_ativo', label: 'Pedido Ativo', type: 'text', readonly: true },
    { key: 'endereco_entrega', label: 'Endereço de Entrega', type: 'textarea' },
  ],

  // Schema de DB próprio do módulo (tabelas prefixadas)
  database: {
    schema: 'logistica_acme',
    migrations: './backend/db/schema.ts',
  },

  // Event listeners
  events: {
    'conversation.resolved': 'events/onConversationResolved',
  },

  // Rotas backend custom (REST)
  routes: './backend/routes/despacho',

  // Configuração/secrets necessárias (validadas no provisioning)
  config: {
    required: ['ACME_ERP_API_URL', 'ACME_ERP_API_KEY'],
    optional: ['ACME_SLACK_WEBHOOK'],
  },
});
```

10.7.3. Action handler: `backend/actions/cancelarPedido.ts`

```
import type { ActionHandler } from '@blossom/sdk';

export const cancelarPedido: ActionHandler = async (payload, ctx) => {
  const { contact, formData } = payload;
  const pedidoId = contact.customFields?.pedido_ativo;

  if (!pedidoId) {
    return {
      status: 'error',
      message: 'Contato não tem pedido ativo registrado',
    };
  }

  try {
    const response = await ctx.http.post(
      `${ctx.config.ACME_ERP_API_URL}/pedidos/${pedidoId}/cancelar`,
      {
        motivo: formData.motivo,
        observacao: formData.observacao,
        canceladoPor: payload.executedBy.userName,
      },
      {
        headers: { Authorization: `Bearer ${ctx.config.ACME_ERP_API_KEY}` },
        timeout: 10_000,
      }
    );

    // Limpa pedido ativo no contato
    await ctx.api.contacts.update(contact.id, {
      customFields: { pedido_ativo: null },
    });

    return {
      status: 'success',
      message: `Pedido #${pedidoId} cancelado com sucesso`,
      privateNote: `Pedido #${pedidoId} cancelado por ${payload.executedBy.userName}. Motivo: ${formData.motivo}. Obs:
${formData.observacao || '—'}.`,
    };
  } catch (err) {
    ctx.logger.error({ err, pedidoId }, 'Falha ao cancelar pedido no ERP');
    return {
      status: 'error',
      message: `Não foi possível cancelar no ERP: ${err.message}`,
    };
  }
};
```

10.7.4. Action handler: `backend/actions/consultarStatus.ts`

```
import type { ActionHandler } from '@blossom/sdk';

export const consultarStatus: ActionHandler = async (payload, ctx) => {
  const pedidoId = payload.contact.customFields?.pedido_ativo;
  if (!pedidoId) {
    return { status: 'error', message: 'Sem pedido ativo' };
  }

  const status = await ctx.http.get(
    `${ctx.config.ACME_ERP_API_URL}/pedidos/${pedidoId}`,
    { headers: { Authorization: `Bearer ${ctx.config.ACME_ERP_API_KEY}` } }
  );

  return {
    status: 'success',
    privateNote: `□ Status pedido #${pedidoId}:
• Situação: ${status.situacao}
• Previsão: ${status.previsao_entrega}
• Última movimentação: ${status.ultima_movimentacao}
• Transportadora: ${status.transportadora} (código ${status.codigo_rastreio})`,
  };
};
```

10.7.5. Rotas backend: `backend/routes/despacho.ts`

```
import type { RouteRegistrar } from '@blossom/sdk';

export const register: RouteRegistrar = (app, ctx) => {
  // Registrar erro de despacho
  app.post('/despacho/erros', async (req, reply) => {
    const { contactId, tipoErro, descricao, fotoKey } = req.body;

    // Valida permissão
    if (!req.user || ![‘admin', 'agent'].includes(req.user.role)) {
      return reply.code(403).send({ error: 'Forbidden' });
    }

    // Cria registro no schema próprio do módulo
    const erro = await ctx.db.insert('erros_despacho', {
      contactId,
      tipoErro,
      descricao,
      fotoKey,
      registradoPor: req.user.id,
    });

    // Dispara webhook pro ERP
    await ctx.queue.add('notify-erp-erro', { erroId: erro.id });

    // Cria mensagem automática na conversa do contato
    const convs = await ctx.api.conversations.list({ contactId, status: 'open' });
    if (convs[0]) {
      await ctx.api.messages.send(convs[0].id, {
        content: `□ Registramos sua ocorrência #${erro.id}. Nossa equipe já está verificando.`,
      });
    }

    // Notifica supervisor
    await ctx.api.notifications.send({
      role: 'supervisor',
      message: `Novo erro de despacho registrado (#${erro.id})`,
      link: `/plugins/logistica-acme/despacho?erro=${erro.id}`,
    });

    return reply.send({ id: erro.id });
  });

  // Listar erros recentes
  app.get('/despacho/erros', async (req, reply) => {
    const erros = await ctx.db.select('erros_despacho', {
      where: { resolvido: false },
      orderBy: 'created_at DESC',
      limit: 100,
    });
    return reply.send({ erros });
  });

  // Presigned URL para upload de foto
  app.post('/despacho/erros/upload-url', async (req, reply) => {
    const key = `logistica-acme/despacho/${Date.now()}-${req.body.filename}`;
    const url = await ctx.storage.getSignedUploadUrl(key, { expiresIn: 300 });
    return reply.send({ url, key });
  });
};
```

10.7.6. DB schema: `backend/db/schema.ts`

```
import { pgTable, uuid, text, timestamp, boolean } from 'drizzle-orm/pg-core';

// Todas as tabelas do módulo ficam no schema 'logistica_acme'
export const errosDespacho = pgTable('erros_despacho', {
  id: uuid('id').primaryKey().defaultRandom(),
  contactId: uuid('contact_id').notNull(),
  tipoErro: text('tipo_erro').notNull(),
  descricao: text('descricao'),
  fotoKey: text('foto_key'),
  registradoPor: uuid('registrado_por').notNull(),
  resolvido: boolean('resolvido').default(false),
  resolvidoPor: uuid('resolvido_por'),
  resolvidoEm: timestamp('resolvido_em'),
  createdAt: timestamp('created_at').defaultNow(),
});
```

10.7.7. Frontend: `frontend/pages/Despacho.tsx`

```
import { useBlossom } from '@blossom/sdk/react';
import { FormErro } from '../components/FormErro';
import { ListaOcorrencias } from '../components/ListaOcorrencias';

export default function DespachoPage() {
  const { api } = useBlossom();
  const { data: erros, refetch } = api.module.useQuery('/despacho/erros');

  return (
    <div className="flex gap-6 h-full p-6">
      <div className="w-1/2">
        <h2 className="text-xl font-bold mb-4">Registrar Ocorrência</h2>
        <FormErro onSuccess={refetch} />
      </div>
      <div className="w-1/2">
        <h2 className="text-xl font-bold mb-4">Ocorrências em Aberto</h2>
        <ListaOcorrencias erros={erros ?? []} />
      </div>
    </div>
  );
}
```

`FormErro.tsx` (resumo):

```
export function FormErro({ onSuccess }: Props) {
  const { api } = useBlossom();
  const [contactId, setContactId] = useState<string>();
  const [tipoErro, setTipoErro] = useState('');
  const [descricao, setDescricao] = useState('');
  const [foto, setFoto] = useState<File>();

  async function submit() {
    // 1. Upload da foto direto pro R2 via presigned URL
    let fotoKey: string | undefined;
    if (foto) {
      const { url, key } = await api.module.post('/despacho/erros/upload-url', {
        filename: foto.name,
      });
      await fetch(url, { method: 'PUT', body: foto });
      fotoKey = key;
    }

    // 2. Registra a ocorrência
    await api.module.post('/despacho/erros', { contactId, tipoErro, descricao, fotoKey });
    onSuccess();
  }

  return (
    <form onSubmit={submit} className="space-y-4">
      <ContactPicker value={contactId} onChange={setContactId} />
      <Select value={tipoErro} onChange={setTipoErro} options={TIPOS_ERRO} />
      <Textarea value={descricao} onChange={setDescricao} />
      <FileInput accept="image/*" onChange={setFoto} capture="environment" />
      <Button type="submit">Registrar</Button>
    </form>
  );
}
```

10.8. Exemplo Completo 2: Módulo "Tênis Sheiba"

Contexto: cliente vende tênis. Bot recomenda produto, conversa até o fechamento. Precisa de:

- Aba "Catálogo" com CRUD de tênis
- Action "Enviar catálogo" que envia cards de produtos pro WhatsApp
- Action "Gerar link de pagamento" que integra com Stripe/Mercado Pago
- Bot que conversa sobre tênis (implementado fora, conecta via webhook)

10.8.1. `module.config.ts`

```
export default defineModule({
  key: 'tenis-sheiba',
  name: 'Sheiba Tênis',
  version: '0.9.0',

  tabs: [
    {
      key: 'catalogo',
      label: 'Catálogo',
      icon: 'ShoeIcon',
      requiresRole: 'admin',
      component: () => import('./frontend/pages/Catalogo'),
    },
    {
      key: 'pedidos',
      label: 'Pedidos',
      icon: 'ShoppingBagIcon',
      component: () => import('./frontend/pages/Pedidos'),
    },
  ],

  actions: [
    {
      key: 'enviar_recomendacoes',
      label: 'Enviar Recomendações',
      icon: 'SparklesIcon',
      form: {
        tenisIds: { type: 'multi-select-custom', source: '/tenis/search', max: 3 },
      },
      handler: 'enviarRecomendacoes',
    },
    {
      key: 'gerar_link_pagamento',
      label: 'Gerar Link de Pagamento',
      icon: 'CreditCardIcon',
      form: {
        tenisId: { type: 'select-custom', source: '/tenis/search' },
        tamanho: { type: 'number', min: 34, max: 48 },
        desconto: { type: 'number', min: 0, max: 30, suffix: '%' },
      },
      handler: 'gerarLinkPagamento',
    },
  ],

  database: {
    schema: 'sheiba',
    migrations: './backend/db/schema.ts',
  },

  contactFields: [
    { key: 'tamanho_pe', label: 'Tamanho', type: 'number' },
    { key: 'marca_preferida', label: 'Marca Preferida', type: 'text' },
    { key: 'ltv', label: 'LTV', type: 'currency', readonly: true, computed: true },
  ],

  config: {
    required: ['STRIPE_SECRET_KEY', 'STRIPE_WEBHOOK_SECRET'],
  },
});
```

10.8.2. Action: gerar link de pagamento

```
export const gerarLinkPagamento: ActionHandler = async (payload, ctx) => {
  const { tenisId, tamanho, desconto } = payload.formData;
  const tenis = await ctx.db.select('tenis').where({ id: tenisId }).one();

  const precoFinal = tenis.preco * (1 - desconto / 100);

  const stripe = new Stripe(ctx.config.STRIPE_SECRET_KEY);
  const session = await stripe.checkout.sessions.create({
    mode: 'payment',
    line_items: [{
      price_data: {
        currency: 'brl',
        product_data: { name: `${tenis.marca} ${tenis.modelo} (tam. ${tamanho})` },
        unit_amount: Math.round(precoFinal * 100),
      },
      quantity: 1,
    }],
    metadata: {
      contactId: payload.contact.id,
      tenisId,
      tamanho: String(tamanho),
    },
    success_url: 'https://sheiba.com.br/obrigado',
    cancel_url: 'https://sheiba.com.br/carrinho',
  });

  // Envia link como mensagem na conversa
  await ctx.api.messages.send(payload.conversation.id, {
    content: `□ Seu link de pagamento: ${session.url}\nValor: R$ ${precoFinal.toFixed(2)}`,
  });

  return {
    status: 'success',
    privateNote: `Link de pagamento gerado. Valor: R$ ${precoFinal.toFixed(2)}. Stripe session: ${session.id}`,
  };
};
```

10.8.3. Webhook Stripe → marcar pedido pago

```
// Rota custom do módulo
app.post('/stripe/webhook', async (req, reply) => {
  const sig = req.headers['stripe-signature'];
  const event = stripe.webhooks.constructEvent(req.rawBody, sig, ctx.config.STRIPE_WEBHOOK_SECRET);

  if (event.type === 'checkout.session.completed') {
    const { contactId, tenisId, tamanho } = event.data.object.metadata;

    await ctx.db.insert('pedidos', {
      contactId,
      tenisId,
      tamanho: Number(tamanho),
      valor: event.data.object.amount_total / 100,
      status: 'pago',
    });

    // Mensagem automática
    const conv = (await ctx.api.conversations.list({ contactId, status: 'open' }))[0];
    if (conv) {
      await ctx.api.messages.send(conv.id, {
        content: `□ Pagamento confirmado! Seu tênis será enviado em até 3 dias úteis.`,
      });
      await ctx.api.conversations.tag(conv.id, { add: ['pagou', 'a-enviar'] });
    }
  }

  return reply.send({ received: true });
});
```

10.9. Developer Experience pra criar módulos

```
# Scaffold
npx blossom module init logistica-acme

# Roda dev mode com hot-reload do módulo
npx blossom dev --module logistica-acme

# Testa módulo isoladamente
npx blossom module test logistica-acme

# Build + deploy
npx blossom module build logistica-acme
npx blossom module deploy logistica-acme --tenant=acme-logistica
```

10.10. Versionamento e Deploy de Módulos

- Módulos têm versão semântica (`1.2.0`)
- Deploy por tenant: `ENABLED_MODULES=logistica-acme@1.2.0,outro@0.5.1`
- Rollback: redeploy com versão anterior
- Feature flag por módulo (habilitar gradualmente features novas)

---

11. API REST, WebSocket e SDK

11.1. Princípios da API

- **REST** seguindo Richardson Maturity Level 2 (recursos + verbos HTTP + status codes)
- **JSON** padrão, camelCase nas chaves
- **Paginação cursor-based** em listas grandes (evita problemas com offset em dados mutáveis)
- **Versionamento:** prefixo `/api/v1/` — breaking changes promovem pra v2
- **Autenticação:**
  - Sessão (cookies HttpOnly) pra app web
  - API Key (Bearer token) pra integrações externas
  - JWT access + refresh pra mobile/SDK
- **Rate limiting:** 60 req/min default, configurável por endpoint
- **Observability:** `X-Request-Id` propagado e retornado

11.2. Endpoints Principais (REST)

Autenticação

```
POST   /api/v1/auth/login                { email, password } → { accessToken, refreshToken, user }
POST   /api/v1/auth/refresh              { refreshToken } → { accessToken }
POST   /api/v1/auth/logout               → 204
POST   /api/v1/auth/2fa/enable           → { qrCode, secret }
POST   /api/v1/auth/2fa/verify           { code } → 204
```

Usuários (Atendentes)

```
GET    /api/v1/users                     → User[]
POST   /api/v1/users                     { email, name, role, teamIds } → User
GET    /api/v1/users/:id                 → User
PATCH  /api/v1/users/:id                 → User
DELETE /api/v1/users/:id                 → 204
GET    /api/v1/users/me                  → User (self)
PATCH  /api/v1/users/me/status           { status: 'online'|'away'|'offline' }
```

Teams

```
GET    /api/v1/teams                    → Team[]
POST   /api/v1/teams                    { name, memberIds } → Team
PATCH  /api/v1/teams/:id                → Team
DELETE /api/v1/teams/:id                → 204
```

Inboxes (Canais)

```
GET    /api/v1/inboxes                  → Inbox[]
POST   /api/v1/inboxes                  { name, channelType, config } → Inbox
PATCH  /api/v1/inboxes/:id              → Inbox
DELETE /api/v1/inboxes/:id              → 204
GET    /api/v1/inboxes/:id/members      → User[]
POST   /api/v1/inboxes/:id/members      { userIds } → 204
```

Contatos

```
GET    /api/v1/contacts?q=&tag=&cursor=  → { items: Contact[], nextCursor? }
POST   /api/v1/contacts                  → Contact
GET    /api/v1/contacts/:id              → Contact (com identities, tags)
PATCH  /api/v1/contacts/:id              → Contact
DELETE /api/v1/contacts/:id              → 204 (soft)
POST   /api/v1/contacts/:id/merge        { targetId } → Contact (unificação)
GET    /api/v1/contacts/:id/conversations → Conversation[]
POST   /api/v1/contacts/:id/tags         { tagIds } → 204
DELETE /api/v1/contacts/:id/tags/:tagId  → 204
POST   /api/v1/contacts/import           (CSV upload) → Job
GET    /api/v1/contacts/:id/export       → JSON (LGPD)
POST   /api/v1/contacts/:id/purge        → 204 (LGPD hard delete)
```

Conversas

```
GET    /api/v1/conversations?status=&assigned=&inbox=&tag=&cursor=
POST   /api/v1/conversations             { contactId, inboxId } → Conversation
GET    /api/v1/conversations/:id         → Conversation
PATCH  /api/v1/conversations/:id         { status, priority, assignedTo, tags }
POST   /api/v1/conversations/:id/assign  { userId?, teamId?, botId? } → Conversation
POST   /api/v1/conversations/:id/resolve → Conversation
POST   /api/v1/conversations/:id/reopen  → Conversation
POST   /api/v1/conversations/:id/snooze  { until } → Conversation
POST   /api/v1/conversations/:id/messages { content, contentType, isPrivateNote } → Message
GET    /api/v1/conversations/:id/messages?cursor= → { items, nextCursor }
```

Bulk Actions

```
POST   /api/v1/conversations/bulk        { ids, action, params }
  // action ∈ { assign, tag, resolve, snooze, delete }
```

Tags

```
GET    /api/v1/tags                      → Tag[]
POST   /api/v1/tags                      { name, color } → Tag
PATCH  /api/v1/tags/:id                  → Tag
DELETE /api/v1/tags/:id                  → 204
```

Canned Responses

```
GET    /api/v1/canned-responses?teamId=  → CannedResponse[]
POST   /api/v1/canned-responses          { shortcut, content, teamId? }
PATCH  /api/v1/canned-responses/:id      → CannedResponse
DELETE /api/v1/canned-responses/:id      → 204
```

Bots

```
GET    /api/v1/bots                      → Bot[]
POST   /api/v1/bots                      { name, webhookUrl, inboxId } → Bot (secret só retorna aqui)
POST   /api/v1/bots/:id/rotate-secret    → { secret }
```

Campanhas

```
GET    /api/v1/campaigns
POST   /api/v1/campaigns                { name, inboxId, audienceFilter, templateId, scheduledAt }
POST   /api/v1/campaigns/:id/launch     → 202
GET    /api/v1/campaigns/:id/report     → { sent, delivered, read, replied, failed }
```

Automações / Rules

```
GET    /api/v1/automations
POST   /api/v1/automations              { name, trigger, conditions, actions, enabled }
PATCH  /api/v1/automations/:id
```

Actions Customizadas (declaradas por módulo)

```
GET    /api/v1/custom-actions?moduleKey= → CustomAction[]
POST   /api/v1/custom-actions/:id/run    { contactId, conversationId?, formData } → Result
GET    /api/v1/action-logs               → ActionLog[]
```

Analytics

```
GET    /api/v1/analytics/overview?from=&to=          → KPIs
GET    /api/v1/analytics/agents?from=&to=             → por atendente
GET    /api/v1/analytics/inboxes?from=&to=            → por inbox
GET    /api/v1/analytics/sla?from=&to=                → cumprimento SLA
GET    /api/v1/analytics/csat?from=&to=               → scores
GET    /api/v1/analytics/export?type=&from=&to=       → CSV download
```

Settings

```
GET    /api/v1/settings/business-hours
PUT    /api/v1/settings/business-hours   { timezone, schedule[] }
GET    /api/v1/settings/outbound-webhooks
POST   /api/v1/settings/outbound-webhooks { url, events, secret }
```

API Pública (API Key)

Mesmos endpoints acima, autenticados via `Authorization: Bearer <api_key>`. API keys têm escopo (read/write) e rate limit dedicado.

11.3. Eventos WebSocket

Conexão: `wss://<tenant>.blossominbox.com/ws?token=<jwt>`

Cliente se inscreve em "rooms" por interesse:

```
emit('subscribe', { rooms: ['inbox:123', 'conversation:456', 'user:me'] })
```

Eventos recebidos pelo cliente

```
// Nova mensagem em qualquer conversa que o usuário participa
'message.created'     : { conversationId, message }

// Mensagem editada/deletada (onde canal suporta)
'message.updated'     : { conversationId, messageId, changes }
'message.deleted'     : { conversationId, messageId }

// Conversa teve mudança
'conversation.created'  : { conversation }
'conversation.updated'  : { conversationId, changes }
'conversation.assigned' : { conversationId, assignedTo, assignedBy }
'conversation.resolved' : { conversationId, resolvedBy }

// Indicador de digitação
'conversation.typing'   : { conversationId, userId, isTyping }

// Atualização de status do atendente
'user.status_changed'   : { userId, status }

// Notificação in-app
'notification.new'      : { notification }

// Evento custom de módulo
'module.<moduleKey>.<event>' : { ... }
```

Eventos emitidos pelo cliente

```
'subscribe'               : { rooms: string[] }
'unsubscribe'             : { rooms: string[] }
'conversation.typing'     : { conversationId, isTyping }
'message.read'            : { conversationId, messageId }
'user.status'             : { status: 'online'|'away'|'offline' }
```

11.4. Contrato do Bot Webhook

Sistema envia POST para `bot.webhookUrl` em cada nova mensagem:

```
POST https://bot-do-cliente.com/blossom
Headers:
  X-Blossom-Signature: sha256=<hmac>
  X-Blossom-Event: message.created

Body:
{
  "conversationId": "uuid",
  "contactId": "uuid",
  "inboxId": "uuid",
  "message": {
    "id": "uuid",
    "content": "Quero cancelar pedido 123",
    "contentType": "text",
    "sender": { "type": "contact" },
    "createdAt": "2026-04-13T12:00:00Z"
  },
  "contact": {
    "name": "João Silva",
    "phone": "+5511999999999",
    "customFields": { "cpf": "123..." }
  },
  "history": [ /* últimas 20 mensagens */ ]
}
```

Bot responde com (síncrono em < 10s, ou assíncrono chamando `/api/v1/.../messages` depois):

```
{
  "message": "Claro, posso ajudar com o cancelamento. Qual o motivo?",
  "contentType": "text",
  "action": null  // ou "handoff" / "resolve" / "tag"
}
```

Formato de `action`:

```
{ "type": "handoff", "team": "suporte", "note": "Cliente pediu cancelamento fora do prazo." }
{ "type": "resolve" }
{ "type": "tag", "tags": ["cancelamento", "urgente"] }
{ "type": "update_contact", "fields": { "customFields.pedido": "123" } }
```

11.5. Contrato de Action Webhook (Módulo → Sistema do Cliente)

Quando atendente executa Action com `webhook_url` externa:

```
POST https://erp-cliente.com/webhook/cancelar-pedido
Headers:
  X-Blossom-Signature: sha256=<hmac>

Body:
{
  "action": "cancelar_pedido",
  "executedBy": { "userId": "uuid", "userName": "Ana" },
  "contact": { "id": "uuid", "customFields": { "cpf": "..." } },
  "conversation": { "id": "uuid" },
  "formData": { "motivo": "Cliente desistiu", "observacao": "..." },
  "executedAt": "2026-04-13T12:00:00Z"
}
```

Resposta esperada:

```
{
  "status": "success" | "error",
  "message": "Pedido cancelado com sucesso",
  "privateNote": "Cancelamento #42 registrado no ERP",  // opcional, vira private note na conversa
  "contactUpdate": { "customFields": { "status_pedido": "cancelado" } }  // opcional
}
```

11.6. SDK `@blossom/sdk` pra Módulos

Interface principal:

```
import { defineModule } from '@blossom/sdk';

export default defineModule({
  key: 'logistica-acme',
  name: 'Módulo Logística ACME',
  version: '1.0.0',

  // Declarações estáticas
  tabs: [/* ... */],
  actions: [/* ... */],
  contactFields: [/* ... */],

  // Hooks (lazy-loaded)
  onInit: async (ctx) => {
    ctx.logger.info('Módulo carregado');
  },

  // Event listeners
  events: {
    'conversation.resolved': async (event, ctx) => {
      await ctx.api.contacts.update(event.contactId, {
        customFields: { lastResolvedAt: new Date() }
      });
    },
    'module.acme.pedido_criado': async (event, ctx) => {
      await ctx.api.messages.send(event.conversationId, {
        content: `Pedido #${event.pedidoId} criado!`
      });
    }
  },

  // Handlers de action
  actionHandlers: {
    cancelar_pedido: async (payload, ctx) => {
      const result = await ctx.http.post('https://erp.../cancelar', payload);
      return { status: 'success', privateNote: `Cancelado #${result.id}` };
    }
  },

  // Rotas backend custom (Fastify plugin)
  routes: async (fastify) => {
    fastify.post('/despacho/erro', async (req, reply) => {
      // ...
    });
  },
});
```

Interface `ctx` disponibiliza:

```
interface ModuleContext {
  tenant: { id: string; slug: string };
  api: BlossomAPIClient;       // read/write seguro, respeita permissions
  http: FetchWrapper;           // chamadas externas com retry, timeout, logging
  logger: Logger;
  config: ModuleConfig;
  secrets: SecretManager;       // acesso a secrets do módulo
  storage: StorageClient;       // upload/download de arquivos
  queue: QueueClient;           // enfileirar jobs
  events: EventEmitter;         // emitir eventos custom
  db: ModuleDBClient;           // schema próprio do módulo (tabelas prefixadas)
}
```

---

12. Sistema de Eventos e Automações

Blossom Inbox é **event-driven** internamente. Todo componente significativo emite eventos, que são consumidos por: workers, módulos custom, automações declarativas, outbound webhooks.

12.1. Taxonomia de Eventos

```
conversation.created
conversation.updated
conversation.assigned
conversation.reopened
conversation.resolved
conversation.snoozed
message.created
message.updated
message.deleted
message.delivered          // canal confirmou entrega
message.read               // canal confirmou leitura
contact.created
contact.updated
contact.merged
contact.tag_added
contact.tag_removed
user.logged_in
user.logged_out
user.status_changed
campaign.launched
campaign.message_sent
campaign.message_failed
automation.triggered
action.executed
module.<key>.<custom>      // eventos custom de módulo
```

12.2. Arquitetura do Event Bus

- Implementação: **Redis Streams** (durável, replay possível)
- Produtor: services do core emitem via `EventBus.emit(eventName, payload)`
- Consumidores:
  - **Worker interno** (webhook delivery, analytics aggregation)
  - **Automation Engine** (rule matching)
  - **Module Event Hooks** (carregados via SDK)
  - **Outbound Webhook Dispatcher** (entrega pra clientes externos)
- Garantias: at-least-once delivery, deduplicação por `eventId`
- Retry: exponential backoff com DLQ (dead letter queue) após 10 tentativas

12.3. Automation Engine

Automações declarativas via UI. Exemplo serializado:

```json
{
  "id": "uuid",
  "name": "Escalar conversa travada pra supervisor",
  "enabled": true,
  "trigger": {
    "type": "schedule",
    "cron": "*/5 * * * *"
  },
  "conditions": {
    "all": [
      { "field": "conversation.status", "op": "eq", "value": "pending" },
      { "field": "conversation.waitingForAgentSince", "op": "gt_minutes", "value": 30 },
      { "field": "conversation.tags", "op": "contains", "value": "urgente" }
    ]
  },
  "actions": [
    { "type": "assign", "params": { "userId": "uuid-supervisor" } },
    { "type": "send_notification", "params": { "userId": "uuid-supervisor", "message": "Conversa urgente travada!" } },
    { "type": "tag", "params": { "add": ["escalada"] } }
  ]
}
```

Triggers suportados:

- `event` — dispara no evento `{eventName}`
- `schedule` — cron
- `condition_change` — quando condição passa a ser verdadeira

12.4. Outbound Webhooks

Cliente configura em settings: URL + eventos subscritos + secret.

Implementação:

- Fila BullMQ dedicada
- Timeout 10s
- Retry: 30s, 2min, 10min, 1h, 6h, 24h (6 tentativas)
- Entrega assinada (`X-Blossom-Signature`)
- Logs de entrega em `webhook_deliveries`
- UI pra reentregar manualmente falhas

12.5. Idempotência

Todo webhook entrante (Twilio, bots) inclui ID externo. Sistema mantém `channel_msg_id UNIQUE` em `messages` — reentregas não duplicam. Eventos internos têm `eventId UUID` pra deduplicação em consumers.

---

13. Wireframes e UX

Wireframes ASCII das telas principais. Versão visual detalhada em Figma (a criar).

13.1. Layout Geral

```
┌──────────────────────────────────────────────────────────────┐
│  [LOGO]  Busca global...          [□3]  [Ana ▼]              │
├──┬───────────────────────────────────────────────────────────┤
│□ │  (conteúdo muda conforme aba selecionada)                 │
│□ │                                                           │
│□ │                                                           │
│□ │                                                           │
│⚙ │                                                           │
│  │                                                            │
│+ │  (abas custom de módulos aparecem aqui com ícones)        │
└──┴───────────────────────────────────────────────────────────┘
 ^ sidebar fixa com abas
```

13.2. Inbox (aba principal)

```
┌──────────────────────────────────────────────────────────────────────┐
│ INBOX                                                                 │
├──────────────┬──────────────────────────────┬──────────────────────┤
│ FILTROS      │ CONVERSAS                    │ CONVERSA SELEC.      │
│              │                              │                      │
│ ▼ Status     │ ┌──────────────────────────┐ │ ╔═══════════════════╗ │
│  ◉ Open      │ │ João Silva     [WA]  2m  │ │ ║ João Silva        ╕ │
│  ○ Pend.     │ │ "Meu pedido não chegou"  │ │ ║ +55 11 9...       ╕ │
│  ○ Resolv.   │ │ □ pedido □ urgente       │ │ ╟───────────────────╢ │
│              │ ├──────────────────────────┤ │ ║ ...histórico      ╕ │
│ ▼ Inbox      │ │ Maria P.       [EM] 15m  │ │ ║                   ╕ │
│  ☑ WA Sup    │ │ "Gostaria de trocar..."  │ │ ║ [João]: Olá       ╕ │
│  ☑ Email     │ ├──────────────────────────┤ │ ║ [Ana]: Bom dia    ╕ │
│              │ │ Carlos       [CHAT] 1h   │ │ ║ [□ foto.jpg]      ╕ │
│ ▼ Assign     │ │ "Quanto custa o..."      │ │ ║                   ╕ │
│  ◉ Mine      │ │                          │ │ ║                   ╕ │
│  ○ Unass.    │ │ ...                      │ │ ╟───────────────────╢ │
│  ○ All       │ └──────────────────────────┘ │ ║ Responder:        ╕ │
│              │                              │ ║ ┌──────────┐        ╕ │
│ ▼ Tags       │                              │ ║ │ [texto]  │        ╕ │
│  □ pedido    │                              │ ║ │          │        ╕ │
│  □ bug       │                              │ ║ └──────────┘        ╕ │
│              │                              │ ║ [□][😀][/] [▶]      ╕ │
└──────────────┴──────────────────────────────┴──────────────────────┘
  ^ Filtros    ^ Lista scrollável              ^ Painel ativo
```

Right side alterna entre:

- **Conversa** (mensagens + composer)
- **Contato** (perfil + custom fields + Actions + tags)

13.3. Painel de Contato (lateral direita, alternável)

```
┌──────────────────────────┐
│ [avatar] João Silva      │
│ +55 11 99999-9999        │
│ joao@email.com           │
├──────────────────────────┤
│ □ pedido  □ urgente [+]  │
├──────────────────────────┤
│ CAMPOS (custom do cliente)│
│ CPF: 123.456.789-00      │
│ Endereço: Rua X, 123     │
├──────────────────────────┤
│ ACTIONS                  │
│ [□ Cancelar Pedido]      │
│ [□ Registrar Ocorrência] │
│ [□ Aplicar Cupom]        │
├──────────────────────────┤
│ HISTÓRICO DE CONVERSAS   │
│ • WA 10/04 (resolvida)   │
│ • EM 03/03 (resolvida)   │
│ • WA 15/02 (resolvida)   │
└──────────────────────────┘
```

13.4. Composer de Mensagem

```
┌──────────────────────────────────────────┐
│ [Escreva sua resposta...]                 │
│                                           │
│                                           │
├──────────────────────────────────────────┤
│ [□ Anexo] [😀 Emoji] [/ Canned] [□ Note]   │
│                              [Agendar▼] [▶]  │
└──────────────────────────────────────────┘
  - [/ canned]: popup com search de templates
  - [□ Note]: toggle pra Private Note mode (fundo amarelo)
  - [Agendar]: scheduled send picker
```

13.5. Dashboard

```
┌──────────────────────────────────────────────────────────┐
│ DASHBOARD                  Período: [Últimos 7 dias ▼]   │
├──────────────────────────────────────────────────────────┤
│ ┌────────┐ ┌────────┐ ┌────────┐ ┌────────┐              │
│ │ 1.234  │ │   89   │ │  3m12s │ │  4.7□  │              │
│ │ Msgs   │ │ Abertas│ │ 1ª Resp│ │ CSAT   │              │
│ └────────┘ └────────┘ └────────┘ └────────┘              │
├──────────────────────────────────────────────────────────┤
│ VOLUME POR DIA                                           │
│ ▂▅▃▇▅▂     (gráfico de linha)                            │
├──────────────────────────────────────────────────────────┤
│ POR ATENDENTE              │ POR CANAL                   │
│ Ana       ▇▇▇▇▇▇▇ 342      │ WhatsApp  ▇▇▇▇▇▇▇ 68%       │
│ Bruno     ▇▇▇▇▇   256      │ Email     ▇▇▇ 21%           │
│ Carla     ▇▇▇     189      │ WebChat   ▇ 11%             │
│ Bot       ▇▇▇▇▇▇▇▇▇ 447    │                             │
└──────────────────────────────────────────────────────────┘
```

13.6. Aba de Contatos

```
┌──────────────────────────────────────────────────────────┐
│ CONTATOS                          [+ Novo] [↑ Import] [↓]  │
├──────────────────────────────────────────────────────────┤
│ [□ Buscar...]   [□ Filtrar tags ▼]  [Canal ▼]  [Ordenar ▼] │
├──────────────────────────────────────────────────────────┤
│ □ │ Nome         │ Canais    │ Tags        │ Última conv  │  │
│ □ │ João Silva   │ WA, EM    │ pedido      │ 2h atrás     │  │
│ □ │ Maria Santos │ EM        │ cliente-vip │ 1d atrás     │  │
│ □ │ Carlos Lima  │ WA        │ -           │ 3d atrás     │  │
│ ...                                                      │
├──────────────────────────────────────────────────────────┤
│ (com seleção)  [Aplicar tag] [Deletar] [Exportar]          │
└──────────────────────────────────────────────────────────┘
```

13.7. Criação de Campanha

```
┌──────────────────────────────────────────────────────────┐
│ NOVA CAMPANHA                                            │
├──────────────────────────────────────────────────────────┤
│ Nome:        [Promo Dia das Mães_______________]         │
│ Inbox:       [WhatsApp Vendas ▼]                         │
│ Enviado por: [Bot: Assistente Vendas ▼]                  │
│                                                          │
│ AUDIÊNCIA                                                │
│  Tags:       [ cliente-vip  x ]  [+ tag]                 │
│  Excluir:    [ optout  x ]                               │
│  Estimativa: 2.341 contatos                              │
│                                                          │
│ TEMPLATE (aprovado Meta)                                 │
│  [promo_dia_maes_v2 ▼]                                   │
│  Pré-visualização:                                       │
│  ┌──────────────────────────────────────┐                │
│  │ Oi, {nome}! □                        │                │
│  │ Promo Dia das Mães até amanhã: {link}  │              │
│  └──────────────────────────────────────┘                │
│                                                          │
│ AGENDAMENTO                                              │
│  ◉ Enviar agora                                          │
│  ○ Agendar [data/hora]                                   │
│                                                          │
│                              [Salvar Rascunho] [□ Lançar] │
└──────────────────────────────────────────────────────────┘
```

13.8. Settings (exemplo: Business Hours)

```
┌──────────────────────────────────────────────────────────┐
│ SETTINGS > Horário Comercial                             │
├──────────────────────────────────────────────────────────┤
│ Timezone: [America/Sao_Paulo ▼]                          │
│                                                          │
│ Dias:        De          Até                             │
│ ☑ Segunda  [09:00]      [18:00]                          │
│ ☑ Terça    [09:00]      [18:00]                          │
│ ☑ Quarta   [09:00]      [18:00]                          │
│ ☑ Quinta   [09:00]      [18:00]                          │
│ ☑ Sexta    [09:00]      [18:00]                          │
│ ☐ Sábado                                                 │
│ ☐ Domingo                                                │
│                                                          │
│ Feriados: [+ Adicionar]                                  │
│   • 07/09/2026  Independência                            │
│   • 12/10/2026  Nossa Senhora                            │
│                                                          │
│ FORA DE HORÁRIO                                          │
│ ☑ Enviar auto-reply                                      │
│   Mensagem: [Olá! Nosso horário de atendimento é seg-sex...]  │
│                                                          │
│                                          [Salvar]        │
└──────────────────────────────────────────────────────────┘
```

13.9. Princípios de UX

1. **Keyboard-first** — power users (atendentes pesados) podem operar quase tudo sem mouse
2. **Real-time sem flicker** — atualizações WebSocket aplicam sem rerender visível da lista
3. **Loading states** com skeleton, nunca tela branca
4. **Errors são acionáveis** — mensagens com "Tentar novamente" em erros de rede
5. **Densidade informacional alta** — atendente precisa ver muito em pouco espaço, mas sem caos
6. **Dark mode** nativo (essencial pra quem fica horas atendendo)
7. **Acessibilidade** — WCAG AA mínimo, navegação por teclado completa
8. **Mobile responsivo** desde o dia 1 (atendentes usam celular também)

---

14. Infra, Deploy e Provisioning

14.1. Arquitetura de Infraestrutura por Fase

Fase 1 — MVP Interno (1-5 tenants)

```
┌──────────────────────────────────────────────┐
│  Hetzner VPS CX32 (€13/mês)                  │
│  ┌──────────────────────────────────────┐    │
│  │  Coolify (self-hosted PaaS)          │    │
│  │  ┌────────┐ ┌────────┐ ┌────────┐    │    │
│  │  │tenant-1│ │tenant-2│ │tenant-3│    │    │
│  │  │(full   │ │(full   │ │(full   │    │    │
│  │  │ stack) │ │ stack) │ │ stack) │    │    │
│  │  └────────┘ └────────┘ └────────┘    │    │
│  │  Traefik (reverse proxy + Let's Encrypt)│  │
│  └──────────────────────────────────────┘    │
└──────────────────────────────────────────────┘
          │
          ├──► Cloudflare R2 (mídia, compartilhado)
          ├──► Backblaze B2 (backups criptografados)
          └──► Sentry + BetterStack (observability)
```

Fase 2-3 — Crescimento (5-50 tenants)

```
┌──────────────────────────────────────────┐
│  Cloudflare (CDN + WAF + DNS)            │
└──────────────────────────────────────────┘
                │
                │
    ┌───────────┴──────────┐
    │   Load Balancer      │
    │  (Hetzner LB/Caddy)  │
    └──────────┬───────────┘
               │           │
  ┌────────────▼──┐  ┌────▼──────────┐
  │ VPS App 1     │  │ VPS App 2     │  ← múltiplas VPS rodando Coolify em cluster
  │ (Coolify)     │  │ (Coolify)     │
  └───────────────┘  └───────────────┘
               │           │
  ┌────────────▼───────────▼──────────┐
  │  Managed Postgres (HA)            │ ← Neon/Supabase paid ou PG próprio com HA
  └───────────────────────────────────┘
```

Fase 4+ — Escala (50+ tenants)

- Kubernetes (k3s self-hosted em múltiplas VPS ou managed)
- Postgres dedicado por tier (shared em free, dedicado em enterprise)
- Multi-região (BR + US opcional)

14.2. Script `provision-tenant.sh` (versão completa)

```bash
#!/usr/bin/env bash
set -euo pipefail

# Uso: ./provision-tenant.sh <slug> <admin-email> [modules]
# Ex:  ./provision-tenant.sh acme admin@acme.com "logistica-acme"

SLUG=$1
ADMIN_EMAIL=$2
MODULES=${3:-""}
DOMAIN="${SLUG}.blossominbox.com"

log() { echo "[$(date +%H:%M:%S)] $*"; }

# ---- 1. Validação ----
log "Validando slug..."
[[ "$SLUG" =~ ^[a-z0-9][a-z0-9-]{2,30}$ ]] || { log "ERRO: slug inválido"; exit 1; }

# ---- 2. DNS (Cloudflare) ----
log "Criando DNS ${DOMAIN}..."
curl -sX POST "https://api.cloudflare.com/client/v4/zones/${CF_ZONE_ID}/dns_records" \
  -H "Authorization: Bearer ${CF_TOKEN}" \
  -H "Content-Type: application/json" \
  -d "{\"type\":\"A\",\"name\":\"${SLUG}\",\"content\":\"${INFRA_IP}\",\"proxied\":true}" \
  > /tmp/cf.json
grep -q '"success":true' /tmp/cf.json || { log "ERRO DNS"; exit 1; }

# ---- 3. Secrets ----
log "Gerando secrets..."
JWT_SECRET=$(openssl rand -base64 48)
ENC_KEY=$(openssl rand -base64 32)
ADMIN_PASSWORD=$(openssl rand -base64 18)
DB_PASSWORD=$(openssl rand -base64 24)

# ---- 4. Postgres ----
log "Criando banco..."
PGPASSWORD=$PG_ADMIN_PASS psql -h "$PG_HOST" -U postgres <<EOF
CREATE USER tenant_${SLUG} WITH PASSWORD '${DB_PASSWORD}';
CREATE DATABASE blossom_${SLUG} OWNER tenant_${SLUG};
GRANT ALL PRIVILEGES ON DATABASE blossom_${SLUG} TO tenant_${SLUG};
EOF

DATABASE_URL="postgresql://tenant_${SLUG}:${DB_PASSWORD}@${PG_HOST}:5432/blossom_${SLUG}"

# ---- 5. Redis namespace ----
REDIS_URL="redis://${REDIS_HOST}:6379/0?prefix=tenant:${SLUG}:"

# ---- 6. R2 bucket prefix ----
R2_PREFIX="tenants/${SLUG}"

# ---- 7. Criar app no Coolify via API ----
log "Criando app no Coolify..."
COOLIFY_APP_ID=$(curl -sX POST "${COOLIFY_URL}/api/v1/applications" \
  -H "Authorization: Bearer ${COOLIFY_TOKEN}" \
  -H "Content-Type: application/json" \
  -d @- <<JSON | jq -r '.id'
{
  "name": "blossom-${SLUG}",
  "git_repository": "${GIT_REPO}",
  "git_branch": "main",
  "build_pack": "dockerfile",
  "dockerfile": "Dockerfile.tenant",
  "domain": "${DOMAIN}",
  "env_vars": {
    "TENANT_SLUG": "${SLUG}",
    "DATABASE_URL": "${DATABASE_URL}",
    "REDIS_URL": "${REDIS_URL}",
    "JWT_SECRET": "${JWT_SECRET}",
    "ENCRYPTION_KEY": "${ENC_KEY}",
    "R2_BUCKET": "${R2_BUCKET}",
    "R2_PREFIX": "${R2_PREFIX}",
    "ENABLED_MODULES": "${MODULES}",
    "SENTRY_DSN": "${SENTRY_DSN_TENANT}",
    "NODE_ENV": "production"
  }
}
JSON
)

# ---- 8. Deploy ----
log "Iniciando deploy..."
curl -sX POST "${COOLIFY_URL}/api/v1/applications/${COOLIFY_APP_ID}/deploy" \
  -H "Authorization: Bearer ${COOLIFY_TOKEN}" > /dev/null

# ---- 9. Aguardar health check ----
log "Aguardando app ficar pronto..."
for i in {1..60}; do
  if curl -sf "https://${DOMAIN}/api/v1/health" > /dev/null; then
    log "App saudável"
    break
```

```
  fi
  sleep 5
done

# ---- 10. Rodar migrations ----
log "Rodando migrations..."
curl -sfX POST "https://${DOMAIN}/api/v1/admin/migrate" \
  -H "X-Admin-Token: ${INFRA_ADMIN_TOKEN}"

# ---- 11. Seed admin user ----
log "Criando usuário admin..."
curl -sfX POST "https://${DOMAIN}/api/v1/admin/seed-admin" \
  -H "X-Admin-Token: ${INFRA_ADMIN_TOKEN}" \
  -H "Content-Type: application/json" \
  -d "{\"email\":\"${ADMIN_EMAIL}\",\"password\":\"${ADMIN_PASSWORD}\"}"

# ---- 12. Registrar no painel central ----
log "Registrando tenant no painel central..."
curl -sfX POST "${CENTRAL_API}/tenants" \
  -H "Authorization: Bearer ${CENTRAL_TOKEN}" \
  -d "{\"slug\":\"${SLUG}\",\"domain\":\"${DOMAIN}\",\"coolifyAppId\":\"${COOLIFY_APP_ID}\"}"

# ---- 13. Summary ----
cat <<SUMMARY

 Tenant ${SLUG} provisionado com sucesso!

URL:      https://${DOMAIN}
Admin:    ${ADMIN_EMAIL}
Senha:    ${ADMIN_PASSWORD}
Módulos:  ${MODULES:-(nenhum)}

Log completo: /var/log/blossom/tenants/${SLUG}.log

SUMMARY
```

14.3. Docker Compose — versão completa

```yaml
version: '3.9'

services:
  backend:
    build:
      context: .
      dockerfile: apps/backend/Dockerfile
    restart: unless-stopped
    environment:
      NODE_ENV: production
      TENANT_SLUG: ${TENANT_SLUG}
      DATABASE_URL: ${DATABASE_URL}
      REDIS_URL: ${REDIS_URL}
      JWT_SECRET: ${JWT_SECRET}
      ENCRYPTION_KEY: ${ENCRYPTION_KEY}
      R2_BUCKET: ${R2_BUCKET}
      R2_PREFIX: ${R2_PREFIX}
      R2_ACCESS_KEY: ${R2_ACCESS_KEY}
      R2_SECRET_KEY: ${R2_SECRET_KEY}
      TWILIO_ACCOUNT_SID: ${TWILIO_ACCOUNT_SID}
      TWILIO_AUTH_TOKEN: ${TWILIO_AUTH_TOKEN}
      POSTMARK_SERVER_TOKEN: ${POSTMARK_SERVER_TOKEN}
      SENTRY_DSN: ${SENTRY_DSN}
      ENABLED_MODULES: ${ENABLED_MODULES}
    labels:
      - "traefik.enable=true"
      - "traefik.http.routers.backend.rule=Host(`${DOMAIN}`) && PathPrefix(`/api`, `/webhooks`, `/ws`)"
      - "traefik.http.routers.backend.tls.certresolver=letsencrypt"
    depends_on:
      postgres: { condition: service_healthy }
      redis: { condition: service_healthy }
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:3000/api/v1/health"]
      interval: 30s
      timeout: 5s
      retries: 3

  worker:
    build:
      context: .
      dockerfile: apps/backend/Dockerfile
    restart: unless-stopped
    command: ["node", "dist/worker.js"]
    environment:
      NODE_ENV: production
      WORKER_CONCURRENCY: 10
      DATABASE_URL: ${DATABASE_URL}
      REDIS_URL: ${REDIS_URL}
      # (demais vars iguais ao backend)
    depends_on:
      redis: { condition: service_healthy }

  frontend:
    build:
      context: .
      dockerfile: apps/frontend/Dockerfile
    restart: unless-stopped
    environment:
      NEXT_PUBLIC_API_URL: https://${DOMAIN}
      NEXT_PUBLIC_WS_URL: wss://${DOMAIN}/ws
      NEXT_PUBLIC_SENTRY_DSN: ${SENTRY_DSN_FRONT}
    labels:
      - "traefik.enable=true"
      - "traefik.http.routers.front.rule=Host(`${DOMAIN}`)"
      - "traefik.http.routers.front.tls.certresolver=letsencrypt"

  postgres:
    image: postgres:16-alpine
    restart: unless-stopped
    environment:
      POSTGRES_DB: blossom
      POSTGRES_USER: blossom
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
    volumes:
      - postgres_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U blossom"]
      interval: 10s
    command: >
      postgres
        -c max_connections=100
        -c shared_buffers=256MB
        -c effective_cache_size=1GB
        -c work_mem=8MB
        -c maintenance_work_mem=64MB
        -c wal_buffers=16MB

  redis:
    image: redis:7-alpine
```

```yaml
    restart: unless-stopped
    volumes:
      - redis_data:/data
    command: >
      redis-server
        --save 60 1000
        --appendonly yes
        --maxmemory 512mb
        --maxmemory-policy allkeys-lru
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 10s

volumes:
  postgres_data:
  redis_data:
```

14.4. GitHub Actions CI/CD (exemplo)

`.github/workflows/ci.yml`:

```yaml
name: CI

on:
  pull_request:
  push:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    services:
      postgres:
        image: postgres:16-alpine
        env:
          POSTGRES_PASSWORD: test
        options: >-
          --health-cmd pg_isready --health-interval 10s --health-timeout 5s --health-retries 5
        ports: ['5432:5432']
      redis:
        image: redis:7-alpine
        ports: ['6379:6379']

    steps:
      - uses: actions/checkout@v4
      - uses: pnpm/action-setup@v2
        with: { version: 9 }
      - uses: actions/setup-node@v4
        with: { node-version: 20, cache: pnpm }

      - run: pnpm install --frozen-lockfile
      - run: pnpm lint
      - run: pnpm type-check
      - run: pnpm test:unit
      - run: pnpm test:integration
        env:
          DATABASE_URL: postgresql://postgres:test@localhost:5432/test
          REDIS_URL: redis://localhost:6379

  build:
    needs: test
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: docker/setup-buildx-action@v3
      - uses: docker/login-action@v3
        with:
          registry: ghcr.io
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}
      - uses: docker/build-push-action@v5
        with:
          context: .
          file: apps/backend/Dockerfile
          push: true
          tags: |
            ghcr.io/blossom-boost/inbox-backend:latest
            ghcr.io/blossom-boost/inbox-backend:${{ github.sha }}
          cache-from: type=gha
          cache-to: type=gha,mode=max

  deploy:
    needs: build
    runs-on: ubuntu-latest
    strategy:
      matrix:
        tenant: [acme, everydayfit, blossom-internal]  # lidos dinamicamente no futuro
    steps:
      - name: Trigger Coolify deploy
        run: |
          curl -X POST "${{ secrets.COOLIFY_URL }}/api/v1/applications/blossom-${{ matrix.tenant }}/deploy" \
            -H "Authorization: Bearer ${{ secrets.COOLIFY_TOKEN }}"
```

14.5. Backups

Postgres:

- Diário: `pg_dump --format=custom --compress=9` → encrypted com `age` → upload pro B2
- Horário: WAL archiving via `pgbackrest` (Fase 2+) pra PITR (point-in-time recovery)
- Retenção: 7 diários + 4 semanais + 12 mensais

Mídia (R2):

- Versionamento nativo habilitado
- Replicação cross-region (Fase 3+)

Config/Secrets:

- Exportados do Coolify semanalmente pra repo privado criptografado (git-crypt ou sops)

Teste de restore:

- Roda mensalmente em staging, valida integridade
- Runbook `docs/runbooks/restore-from-backup.md`

Script `scripts/backup-tenant.sh`:

```bash
#!/usr/bin/env bash
set -euo pipefail
SLUG=$1
TS=$(date -u +%Y%m%dT%H%M%SZ)
FILE="/tmp/blossom-${SLUG}-${TS}.dump"

# Dump
PGPASSWORD=$PG_PASS pg_dump \
  --host=$PG_HOST --username=tenant_${SLUG} \
  --format=custom --compress=9 \
  --file=$FILE \
  blossom_${SLUG}

# Criptografar
age -r $AGE_PUBLIC_KEY -o ${FILE}.age $FILE
rm $FILE

# Upload B2
rclone copy ${FILE}.age b2:blossom-backups/tenants/${SLUG}/
rm ${FILE}.age

echo "Backup ${SLUG} concluído: ${TS}"
```

Rodar via cron: `0 3 * * * /opt/blossom/scripts/backup-all-tenants.sh`

14.6. Disaster Recovery

Objetivos:

- **RTO** (tempo pra restaurar): 1h pra MVP, 15min pra produção
- **RPO** (perda aceitável): 24h MVP, 15min produção (com WAL streaming)

Cenários e playbooks:

Cenário 1: VPS morre

1. Provisionar nova VPS (Terraform/script — 5min)
2. Instalar Coolify (10min)
3. Restaurar volumes de backup
4. Redeploy apps
5. Atualizar DNS se IP mudou
6. **Total: ~45min**

Cenário 2: Postgres corrompido de 1 tenant

1. Parar backend+worker do tenant
2. Drop database corrompido
3. Restore do último dump (diário) + aplicar WAL se disponível
4. Iniciar apps
5. Smoke test
6. **Total: ~20min**

Cenário 3: R2 perde bucket

1. Contato Cloudflare support (suporte de enterprise SLA)
2. Restore da replica cross-region
3. Se falhar: mídias perdidas, conversas funcionais (placeholder "mídia indisponível")

Cenário 4: Comprometimento de secret

1. Rotacionar secret afetado (JWT, API keys)
2. Invalidar todas as sessões (`redis FLUSHDB`)
3. Forçar re-login de todos atendentes
4. Audit log review
5. Post-mortem obrigatório

14.7. Monitoramento e Alertas (Runbook)

Canal de alertas: Slack + email + SMS (críticos).

Alertas P0 (acordam alguém):

- Tenant offline > 2min
- Error rate > 10% por 5min
- Banco inacessível
- Vazamento de credencial detectado

Alertas P1 (próximo dia útil):

- Error rate > 2% por 15min
- Latência p95 > 2s por 30min
- Fila com > 5000 jobs pendentes
- Disco > 80%
- Certificado SSL expirando em < 7 dias
- Backup falhou

Alertas P2 (semanal):

- CPU > 70% sustentado
- Memory > 80% sustentado
- Taxa de erros em módulo custom

14.8. Runbook: Responder Alerta "Tenant Offline"

```
1. Verificar status:
   - curl https://<tenant>.blossominbox.com/api/v1/health
   - Coolify UI → app status

2. Se container parado:
   - Ver logs: coolify logs blossom-<tenant> --tail 100
   - Diagnosticar causa (OOM, crash loop, config inválida)
   - Reiniciar: coolify restart blossom-<tenant>

3. Se container rodando mas unhealthy:
   - Ver conexão DB: docker exec blossom-<tenant>-backend psql $DATABASE_URL -c "\l"
   - Ver Redis: docker exec blossom-<tenant>-backend redis-cli ping

4. Se DB inacessível:
   - Escalate pra P0
   - Considerar fallback (read-only mode)

5. Comunicar cliente se downtime > 10min

6. Post-mortem se > 30min
```

14.9. CI de Segurança

- **Dependabot** — atualizações de segurança automáticas
- **Snyk** — scan de vulnerabilidades em dependências e Docker images
- **SAST** (Semgrep) — análise estática de padrões inseguros
- **Secrets scanning** (Gitleaks) — pre-commit hook + CI

---

15. Segurança

15.1. Autenticação

- Password hashing com **Argon2id**
- JWT de sessão curta (15 min) + refresh token (7 dias, rotativo)
- 2FA opcional (TOTP via `otplib`)
- Rate limiting agressivo em `/auth/*` (5 tentativas/min por IP)

15.2. Autorização

- RBAC (`admin`, `supervisor`, `agent`)
- Hooks `onRequest` do Fastify validam auth + permissão em cada rota protegida
- Middleware valida que atendente tem acesso ao inbox da conversa

15.3. Criptografia

- **Em trânsito:** TLS 1.3 obrigatório, HSTS
- **Em repouso:**
  - Campos sensíveis (`inboxes.config` com tokens Twilio/IMAP) criptografados com AES-256-GCM
  - Chave mestre por instância, gerenciada via Coolify secrets

15.4. Validação de webhooks

- Twilio: validar `X-Twilio-Signature`
- Postmark: HMAC com secret
- Telegram: secret token no URL
- Bots customizados: HMAC SHA256

15.5. LGPD

- Campo `deleted_at` — soft delete, purge automático após 90 dias
- Endpoint `DELETE /contacts/:id/purge` — hard delete sob solicitação do titular
- Consentimento: formulário obrigatório antes de adicionar contato em campanha
- Export de dados do titular: `GET /contacts/:id/export` (JSON completo)
- Nomear DPO por cliente (parte do processo de onboarding SaaS)

15.6. Auditoria

- `audit_logs` registra toda ação sensível (assign de conversa, alteração de contato, execução de action, envio de campanha)
- Retenção mínima: 2 anos

15.7. Checklist OWASP Top 10

- □ Injection: ORM (Drizzle) parametriza queries
- □ Broken Auth: Better Auth testado em produção
- □ Sensitive Data: criptografia em repouso
- □ XXE: não processamos XML de usuário
- □ Broken Access Control: guards + testes
- □ Misconfiguration: secrets via env, config review em PR
- □ XSS: React escapa por default + CSP headers
- □ Deserialization: nunca desserializar input não confiável
- □ Dependências vulneráveis: Dependabot + `npm audit` em CI
- □ Logging: Pino + rotação + Sentry pra erros

---

16. Observabilidade e Monitoramento

16.1. Métricas

- **App:** requests/sec, latência p50/p95/p99 por endpoint, error rate
- **Negócio:** mensagens/dia, novas conversas, SLA de primeira resposta, conversas em aberto
- **Infra:** CPU, memória, disco, conexões Postgres/Redis

16.2. Alertas

- Error rate > 2% por 5min → Sentry + notificação
- Latência p95 > 1s por 10min
- Fila BullMQ com > 1000 jobs pendentes
- Disco > 80%

16.3. Logs

- Estruturados em JSON (Pino)
- `correlationId` propagado em toda a request
- Shipping pra BetterStack ou Loki

16.4. Tracing

- OpenTelemetry + Jaeger (Fase 2+)
- Vai ajudar a debugar latência de webhooks externos

17. Performance e Escalabilidade

17.1. Alvos de Performance

Operação	Alvo p50	Alvo p95	Alvo p99
API GET (conversa, contato)	80ms	250ms	500ms
API POST mensagem	150ms	400ms	800ms
Webhook Twilio → render atendente	500ms	2s	4s
Search global (10k contatos)	100ms	300ms	600ms
Dashboard load	500ms	1.5s	3s

17.2. Estratégia de Cache

- **Redis cache layer** em endpoints read-heavy:
  - Lista de inboxes por user: TTL 5min
  - Lista de tags: TTL 10min
  - Permissions por user: TTL 5min, invalidado em change
  - Dashboard aggregates: TTL 60s
- **Cache-aside** pattern (read-through com refresh on miss)
- **HTTP caching** com ETag em endpoints GET imutáveis

17.3. Otimização do Postgres

- **Índices:**
  - `(assigned_to, status, updated_at DESC)` em `conversations`
  - `(inbox_id, status, last_message_at DESC)` em `conversations`
  - `(conversation_id, created_at)` em `messages`
  - GIN em `contacts.custom_fields` pra search JSONB
  - `tsvector` em `messages.content` pra full-text search
- **Particionamento** (Fase 3+): particionar `messages` por mês se volume > 10M linhas
- **Connection pooling:** PgBouncer em modo transaction
- **Read replicas** (Fase 3+) pra queries de dashboard e analytics
- **Vacuum agressivo** em tabelas com muita UPDATE/DELETE

17.4. Otimização de Mensageria Real-Time

- **Redis pub/sub** entre instâncias de backend pra broadcast de eventos
- **Socket.IO com adapter Redis** pra multi-node
- **Compressão WebSocket** (perMessageDeflate)
- **Batching** de eventos de typing (throttle 500ms)

17.5. Estratégia de Upload de Mídia

- **Direct upload** pro R2/S3 com presigned URL (não passa pelo backend)
- **Backend só registra metadata** após upload completar
- **Transcodificação** (áudio, vídeo) em worker async
- **Thumbnails** gerados em worker, cacheados em R2

17.6. Escala Horizontal

- **Backend stateless** — N instâncias atrás de load balancer (Caddy/Traefik)
- **Session state** em Redis (não em memória do processo)
- **Worker pool** dimensionado separadamente do API
- **Sticky sessions** pra WebSocket (ou usar adapter Redis)

17.7. Escala por Tenant

Porte tenant	Instância recomendada	Msgs/mês
Micro (< 1k msgs/mês)	0.5 vCPU, 1GB RAM	< 1k
Small (1-10k)	1 vCPU, 2GB RAM	1-10k
Medium (10-100k)	2 vCPU, 4GB RAM	10-100k
Large (100k-1M)	4 vCPU, 8GB RAM + read replica	100k-1M
Enterprise (> 1M)	Cluster dedicado	> 1M

17.8. Load Testing

- **k6** scripts cobrindo:
  - 100 atendentes conectados via WebSocket
  - 50 msgs/segundo entrantes
  - 200 req/s no dashboard
- Rodar antes de lançar pra SaaS pago (Fase 4)

17.9. Degradação Graciosa

- **Circuit breaker** em chamadas externas (Twilio, bots)
- **Timeout agressivo** (10s em webhooks externos)
- **Fallback** — se Redis cair, serve de Postgres direto (com log de alerta)
- **Rate limit response 429** com `Retry-After` header

---

18. Boas Práticas de Engenharia

18.1. Código

- **ESLint + Prettier** com preset compartilhado
- **Convenção:** kebab-case pra arquivos, PascalCase pra classes, camelCase pra variáveis
- **Commits:** Conventional Commits (`feat:` , `fix:` , `chore:`)

- **Branch:** trunk-based, feature flags pra long-running work
- **PRs:** review obrigatório, CI verde obrigatório

18.2. Testing

Tipo	Ferramenta	Cobertura alvo
Unit	Vitest	> 70% nos serviços
Integration	Vitest + Testcontainers	fluxos críticos
E2E	Playwright	top 10 user journeys
Load	k6	antes de lançar SaaS

18.3. Documentação

- **README.md** em cada app/package
- **API docs:** OpenAPI auto-gerado + Swagger UI em `/docs` (dev/staging)
- **Storybook** pra `packages/ui`
- **Guia de criação de módulo** detalhado

18.4. Developer Experience

- `pnpm dev` sobe tudo (backend, frontend, worker, postgres, redis) em um comando
- Hot reload em tudo
- Seed data realista (fake contatos/conversas) pra desenvolvimento
- Env de staging espelhando produção

18.5. Code review

- Checklist no template de PR
- No "LGTM" sem contexto — revisor tem que entender a mudança
- Mudanças de schema exigem análise de migration reversível

---

19. Migração e Import de Dados

19.1. Cenários de Migração

1. **Cliente interno vindo de planilha/WhatsApp puro** — import manual ou CSV
2. **Cliente externo vindo de Chatwoot** — migração de contatos + histórico
3. **Cliente externo vindo de Zendesk/Intercom** — import via API deles
4. **Cliente externo já em outro Blossom Inbox** — export completo via API

19.2. Formatos de Import

CSV de Contatos

```
name,email,phone,tags,custom_cpf,custom_endereco
João Silva,joao@email.com,+5511999999999,"cliente-vip;pedido","123","Rua X"
```

Endpoint: `POST /api/v1/contacts/import` — upload CSV, retorna Job ID. Processing em background, atendente acompanha progresso na UI.

JSON Completo (export/import)

```
{
  "version": "1.0",
  "exportedAt": "2026-04-13T...",
  "contacts": [...],
  "conversations": [...],
  "messages": [...],
  "tags": [...],
  "teams": [...]
}
```

19.3. Migração Chatwoot → Blossom Inbox

Script dedicado `scripts/migrate-from-chatwoot.ts`:

1. Conecta no Postgres do Chatwoot (read-only)
2. Mapeia entidades Chatwoot → Blossom:
   - `contacts` → `contacts` + `contact_identities`
   - `conversations` → `conversations`
   - `messages` → `messages` (preservando `channel_msg_id`)
   - `labels` → `tags`
   - `teams` → `teams`
3. Migra mídias: baixa URLs e reupload pro R2
4. Dry-run mode: gera relatório sem escrever
5. Incremental: roda múltiplas vezes sem duplicar (checksum)

19.4. Preservação de IDs Externos

- `channel_msg_id` mantido igual (idempotência em webhooks reentrantes)
- Mapeamento de `external_id` ↔ `blossom_id` em tabela `migration_map` (temporária)
- Permite rollback da migração por 30 dias

19.5. Downtime da Migração

- **Janela de corte**: 2-4h em horário não-comercial
- Durante: Chatwoot em modo read-only, Blossom ainda não recebe webhooks
- Após: rebind dos webhooks dos canais pro Blossom, teste smoke, go-live
- Runbook detalhado em `docs/runbooks/migration-chatwoot.md`

---

20. Modelo de Negócio e Go-to-Market

20.1. Posicionamento

Alternativa brasileira enxuta ao Chatwoot com customização feita pra você.

Versus concorrência:

	Blossom Inbox	Chatwoot Cloud	Zendesk	Intercom	Take Blip	ManyChat
Origem	BR	Índia (open-source)	USA	USA	BR	USA
Preço mensal (10 atend.)	~R$300-600	US$79 (~R$400)	US$690 (~R$3.500)	US$745 (~R$3.800)	R$800+	US$45+
Customização "sob medida"	□ Core do produto	×	□ enterprise	□ enterprise	□ (serviços)	×
Multi-canal	□	□	□	□	□	parcial
Suporte em PT-BR	□	×	□	×	□	×
Deploy dedicado por cliente	□	× cloud	□	×	×	×
Integração WhatsApp	□ (Twilio)	□	□	□	□ nativo	□
LGPD-ready	□ (dados BR)	× (USA)	□	×	□	×

20.2. ICP (Ideal Customer Profile)

Fase 4 — primeiro ICP:

- PMEs brasileiras, 50-500 funcionários
- 3-20 atendentes de suporte/vendas
- Volume: 5k-50k mensagens/mês
- Setores: e-commerce, logística, serviços B2B, educação
- Pain point: Chatwoot é complicado/genérico, Intercom é caro, querem algo "feito pra eles"
- Orçamento: R$500-R$3.000/mês em ferramenta

20.3. Modelo de Preços

Pricing tiered por atendente, com módulos custom como serviço:

Plano	Atendentes	Mensagens/mês	Canais	Módulos custom	Preço
Starter	até 3	10k	WA + Email	—	R$297/mês
Growth	até 10	50k	Todos	até 1 incluído	R$697/mês
Business	até 25	200k	Todos + API	até 3 incluídos	R$1.497/mês
Enterprise	ilimitado	custom	Todos	ilimitado	sob consulta

Add-ons:

- Atendente adicional: R$49/mês
- Mensagem adicional (10k): R$97/mês

- Módulo custom: R$4.000-R$15.000 setup + R$297/mês manutenção
- Onboarding/migração: R$1.500-R$5.000

20.4. Unit Economics (Fase 4+)

Premissas conservadoras:

- Ticket médio: R$800/mês
- Custo de infra por cliente: R$30-80/mês (VPS + storage + Twilio markup)
- Custo de aquisição (CAC): R$2.400 (3x MRR)
- Margem bruta: ~90%
- LTV (24 meses médio): R$19.200
- LTV/CAC: 8x (saudável)

20.5. Canais de Aquisição

Curto prazo (Fase 4):

1. **Outbound** pra empresas da rede dos sócios
2. **Case studies** das empresas da holding Blossom Boost
3. **Conteúdo técnico** em PT-BR (blog, YouTube) sobre atendimento + WhatsApp Business

Médio prazo: 4. **Parcerias** com agências de marketing/consultorias 5. **Integrações destacadas** (Shopify, Nuvemshop, RD Station) 6. **SEO** — "alternativa chatwoot", "chatwoot hospedado brasil" 7. **Comunidade dev** — open-source de partes não-core (SDK, exemplos)

20.6. Roadmap de Vendas (Fase 4)

- **Mês 1-3:** 3 clientes beta (empresas da rede pessoal) com desconto 50%
- **Mês 4-6:** 10 clientes pagantes, refinar onboarding
- **Mês 7-12:** 30 clientes, CAC estabilizado
- **Ano 2:** 100 clientes, breakeven operacional
- **Ano 3:** 300 clientes, profitability

20.7. Métricas de Sucesso

KPI	Alvo Ano 1	Alvo Ano 2
MRR	R$15k	R$80k
Clientes ativos	20	100
Churn mensal	< 5%	< 3%
NPS	> 40	> 50
CSAT plataforma	> 4.5/5	> 4.7/5
Uptime	99.5%	99.9%
Tempo médio onboarding	< 7 dias	< 3 dias

20.8. Estratégia de Open-Source (Opcional, Decisão Pendente)

Modelo inspirado no PostHog/Sentry:

- **Core fechado** (toda a plataforma de atendimento)
- **SDK e exemplos open-source** — atrai dev community, facilita construção de módulos
- **Módulos de integração open** — os de uso geral (Shopify, etc.) podem ser OSS

Beneficio: credibilidade técnica + pipeline de talentos + visibilidade em comunidades dev BR. Risco: código de diferencial vazando. Mitigação: core fica fechado.

---

21. Roadmap em Fases

Fase 0 — Fundação (semanas 1-2)

- [ ] Setup monorepo (Turborepo + pnpm)
- [ ] Docker Compose dev
- [ ] CI básico (lint + test)
- [ ] Schema banco + primeiras migrations
- [ ] Auth + RBAC funcional
- [ ] Deploy pipeline pro Coolify

Fase 1 — MVP Core (semanas 3-8)

- [ ] Inbox com listagem e filtros
- [ ] Conversa: envio/recepção de mensagens
- [ ] WhatsApp via Twilio (send + receive + mídia)
- [ ] WebSocket para real-time
- [ ] Assign de conversa + filtros mine/unassigned
- [ ] Private Notes
- [ ] Contatos: CRUD + tags
- [ ] Bot via webhook (primeira versão)
- [ ] Handoff bot↔humano
- [ ] Primeira instância rodando pra Blossom Boost interna

Fase 2 — Extensibilidade e Canais (semanas 9-14)

- [ ] Sistema de módulos customizáveis (SDK + loader)
- [ ] Módulo piloto: EverydayFit ou logística
- [ ] Actions customizáveis via webhook
- [ ] Campos customizados no contato
- [ ] Email (Postmark Inbound + Send)
- [ ] WebChat Widget
- [ ] Dashboard básico (métricas principais)
- [ ] Provisioning automatizado

Fase 3 — Crescimento (semanas 15-22)

- [ ] Instagram DM + Messenger via Twilio
- [ ] Telegram nativo
- [ ] Campanhas (templates + disparo em massa + relatórios)
- [ ] Dashboard avançado + export CSV
- [ ] Audit log completo
- [ ] 2FA

Fase 4 — Diferenciais SaaS (semanas 23-30)

- [ ] Unificação cross-canal de contatos (heurística + IA)
- [ ] Data enrichment
- [ ] Sugestões de resposta via IA (Claude API)
- [ ] Self-service signup + billing (Stripe)
- [ ] Onboarding automatizado pra novos clientes SaaS
- [ ] Landing page de vendas

Fase 5 — Escala

- [ ] Migração para Kubernetes
- [ ] Múltiplas regiões
- [ ] Tiers (Free/Starter/Pro/Enterprise)
- [ ] Suporte white-label

---

22. Estimativa de Esforço

Assumindo **1 desenvolvedor full-time** (Felipe):

Fase	Duração	Entrega
Fase 0	2 semanas	Fundação pronta
Fase 1	6 semanas	MVP em produção interna
Fase 2	6 semanas	Primeiros módulos custom
Fase 3	8 semanas	Canais extra + campanhas
Fase 4	8 semanas	Pronto para vender SaaS
Total	**~30 semanas (7 meses)**	Produto comercializável

Com **2 devs** (Felipe + mais um), corta pra ~4-5 meses.

Observações:

- Estimativas pressupõem stack que a equipe já domina (Node/TS/Postgres/Next)
- Não inclui descoberta de produto com clientes externos (fase 4 precisa disso paralelamente)

- Buffer de 20% pra imprevistos **não está incluído** — adicionar na prática

---

23. Riscos e Mitigações

Risco	Probabilidade	Impacto	Mitigação
Homologação Twilio/Meta demora	Média	Alto	Começar processo no dia 1, usar número sandbox Twilio pra dev
Escopo expandir sem controle	Alta	Alto	PRD congelado por fase, features novas vão pro backlog
Complexidade do sistema de módulos	Alta	Médio	Começar minúsculo (só abas), evoluir incrementalmente
Cliente logística demandar customização extensa	Alta	Médio	Cobrar separadamente por módulo custom ou incluir no contrato
WhatsApp ban em campanhas	Média	Alto	Só usar templates aprovados Meta, warmup de número, respeitar opt-out
Custo de infra explodir com N clientes	Média	Médio	Começar tudo numa VPS só, monitorar custos, otimizar Redis/PG
LGPD / compliance	Baixa	Alto	Consultoria jurídica antes de Fase 4 (comercialização)
Dependência de Twilio	Média	Alto	Abstrair camada de canal — trocar provedor depois se precisar
Bug crítico em produção sem redundância	Média	Alto	Testes + feature flags + deploy gradual

---

24. Decisões Pendentes

Pontos que **precisam ser confirmados** antes ou durante Fase 0:

1. **Actions customizáveis — hospedagem da lógica:** webhook pro sistema do cliente OU código dentro do módulo custom? (Recomendação: **ambos suportados** — action declara `handler` local OU `webhook_url` externa)
2. **Object Storage:** Cloudflare R2 (recomendado para economia de egress) vs MinIO self-hosted (sem dependência externa). **Recomendação:** R2 pra começar, migrar se necessário.
3. **Email provider:** Postmark (pago, simples) vs Resend (mais moderno) vs IMAP próprio (controle total). **Recomendação:** Postmark Inbound + Send.
4. **Quem administra instâncias dos clientes?** O próprio cliente tem acesso ao Coolify ou só a equipe Blossom? **Recomendação:** só Blossom até Fase 4.
5. **Pricing do SaaS** (Fase 4): por atendente? Por volume de mensagens? Tiered? **A decidir com base em pesquisa de mercado brasileiro.**
6. **Nome oficial do produto:** "Blossom Inbox"? Outro? **Registrar INPI antes de lançar.**
7. **Modelo de suporte:** quem atende clientes do SaaS? SLA? **Definir em Fase 4.**

8. **Open source parcial?** Core aberto, módulos fechados? **Modelo Sentry/PostHog?** Atrai desenvolvedores, mas complica. **A decidir.**

---

25. Anexos

25.1. Glossário

- **Action:** operação disparada pelo atendente que executa lógica custom (cancelar pedido, aplicar cupom, etc.), via webhook ou handler local
- **Agent / User:** atendente da plataforma
- **Assign:** atribuição de conversa a atendente, team ou bot
- **Bot:** entidade no sistema que atende conversas via webhook — plataforma delega, bot implementa lógica
- **Bulk action:** operação aplicada a múltiplas conversas/contatos de uma vez
- **Campaign:** envio em massa de mensagens (email ou WhatsApp via templates aprovados)
- **Canned response:** resposta pré-salva acessível por atalho (ex: `/boasvindas`)
- **Channel:** canal de comunicação (WhatsApp, email, Telegram, etc.)
- **Contact:** pessoa externa que conversa com o negócio
- **Conversation:** conjunto de mensagens trocadas entre um contato e o negócio num inbox
- **CSAT:** Customer Satisfaction — pesquisa de satisfação pós-resolução (escala 1-5)
- **Handoff:** transferência de conversa entre bot e humano
- **Inbox:** canal configurado no sistema (ex: "WhatsApp Suporte", "Email Vendas")
- **Macro:** sequência de ações executável com um clique
- **Mention:** menção `@usuário` em Private Note, gera notificação
- **Module:** pacote de customização instalado numa instância (contém abas, actions, rotas)
- **Private Note:** mensagem interna entre atendentes, invisível ao contato
- **Round-robin:** distribuição automática balanceada de conversas em um team
- **SLA:** Service Level Agreement — metas de tempo de primeira resposta/resolução
- **Snooze:** silenciar conversa por período, reabrir automaticamente
- **Tag / Label:** marcador multi-valor em contato ou conversa
- **Team:** agrupamento de atendentes (ex: Suporte, Vendas)
- **Tenant:** cliente da plataforma (empresa da holding ou SaaS externo) — cada tenant tem sua própria instância
- **Twilio WABA:** WhatsApp Business Account provisionada via Twilio
- **Webhook (entrante):** URL que recebe eventos de canais externos
- **Webhook (saída):** URL configurada pelo cliente para receber eventos do Blossom

25.2. Referências Externas

Produtos de referência

- Chatwoot — inspiração principal
- Intercom — UX reference
- Zendesk — enterprise reference
- Take Blip — concorrente BR

APIs e integrações

- Twilio Conversations API
- Twilio WhatsApp
- Postmark Inbound
- Resend
- Telegram Bot API

Stack técnica

- Fastify Docs
- @fastify/swagger
- @fastify/websocket
- Next.js App Router
- Drizzle ORM
- BullMQ
- Better Auth
- shadcn/ui

Infra

- Coolify
- Dokploy
- Cloudflare R2
- Hetzner Cloud

Compliance e legal

- Lei Geral de Proteção de Dados (LGPD)
- Meta Business Platform Terms
- OWASP Top 10

25.3. Convenções do Documento

- **F#, C#, A#, B#, etc.** — códigos de features referenciáveis em issues/PRs
- **p50 / p95 / p99** — percentis de latência
- **RPO / RTO** — Recovery Point / Time Objective (disaster recovery)
- **MRR / ARR** — Monthly / Annual Recurring Revenue
- **CAC / LTV** — Customer Acquisition Cost / Lifetime Value
- **ICP** — Ideal Customer Profile

25.4. Changelog deste Documento

Versão	Data	Notas
0.1	2026-04-12	Primeira versão, escopo completo
0.2	2026-04-13	Expansão: personas, APIs, eventos, wireframes, performance, migração, go-to-market
0.3	2026-04-13	Backend framework: NestJS → Fastify (dev solo, ramp-up mais rápido, TS nativo)

Documento vivo — revisar ao final de cada fase. Última atualização: Abril 2026